

Assignment1

February 28, 2025

1 IN3050/IN4050 Mandatory Assignment 1: Traveling Salesman Problem - Oskar Ekeid Idland (oskarei)

1.1 Introduction

1.1.1 Rules

Before you begin the exercise, review the rules at this website: <https://www.uio.no/english/studies/examinations/compulsory-activities/mn-ifi-mandatory.html> (This is an individual assignment. You are not allowed to deliver together or copy/share source-code/answers with others.)

Especially, notice that you are **not allowed to use code or parts of code written by others** in your submission. We do check your code against online repositories, so please be sure to **write all the code yourself**. Read also the “Routines for handling suspicion of cheating and attempted cheating at the University of Oslo”: <https://www.uio.no/english/studies/examinations/cheating/index.html> By submitting this assignment, you confirm that you are familiar with the rules and the consequences of breaking them.

Delivery Deadline: Friday, February 28 2025, 23:59

Your submission should be delivered in Devilry. You may redeliver in Devilry before the deadline, but include all files in the last delivery, as only the last delivery will be read. You are recommended to upload preliminary versions hours (or days) before the final deadline.

1.1.2 What to deliver?

Deliver one single zipped folder (.zip, .tgz or .tar.gz) which includes: * PDF report containing: * Your name and username (!) * Instructions on how to run your program, with example runs. * Answers to all questions from assignment. * Brief explanation of what you’ve done. * *Your PDF may be generated by exporting your Jupyter Notebook to PDF, if you have answered all questions in your notebook* * Source code * Source code may be delivered as jupyter notebooks or python files (.py) * The european cities file so the program will run right away. * Any files needed for the group teacher to easily run your program on IFI linux machines.

Important: * Include example runs of your code by doing the reports described in the tasks. Simply implementing the code, but never running it will not give many points. * Include the code that was used to make all reports. Do not include reports of performance and time without also including the code that was used to produce it. * If you weren’t able to finish the assignment, use the PDF report to elaborate on what you’ve tried and what problems you encountered. Students

who have made an effort and attempted all parts of the assignment will get a second chance even if they fail initially. This exercise will be graded PASS/FAIL.

1.1.3 Introduction to exercise

In this exercise, you will attempt to solve an instance of the traveling salesman problem (TSP) using different methods. The goal is to become familiar with evolutionary algorithms and to appreciate their effectiveness on a difficult search problem. You have to use Python to solve the assignment. You must write your program from scratch (but you may use non-EA-related libraries).

	Barcelona	Belgrade	Berlin	Brussels	Bucharest	Budapest
Barcelona	0	1528.13	1497.61	1062.89	1968.42	1498.79
Belgrade	1528.13	0	999.25	1372.59	447.34	316.41
Berlin	1497.61	999.25	0	651.62	1293.40	1293.40
Brussels	1062.89	1372.59	651.62	0	1769.69	1131.52
Bucharest	1968.42	447.34	1293.40	1769.69	0	639.77
Budapest	1498.79	316.41	1293.40	1131.52	639.77	0

Figure 1: First 6 cities from csv file.

1.1.4 Problem

The traveling salesman, wishing to disturb the residents of the major cities in some region of the world in the shortest time possible, is faced with the problem of finding the shortest tour among the cities. A tour is a path that starts in one city, visits all of the other cities, and then returns to the starting point. The relevant pieces of information, then, are the cities and the distances between them. In this instance of the TSP, a number of European cities are to be visited. Their relative distances are given in the data file, *europecancities.csv*, found in the zip file with the mandatory assignment.

(You will use permutations to represent tours in your programs. The **itertools** module in Python provides a permutations function that returns successive permutations, this is useful for exhaustive search)

1.1.5 Helper code for visualizing solutions

Here follows some helper code that you can use to visualize the plans you generate. These visualizations can **help you check if you are making sensible tours or not**. The optimization algorithms below should hopefully find relatively nice looking tours, but perhaps with a few visible inefficiencies.

```
[ ]: import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
np.random.seed(57)
#Map of Europe
europe_map = plt.imread('map.png')
```

```

#Lists of city coordinates
city_coords = {
    "Barcelona": [2.154007, 41.390205], "Belgrade": [20.46, 44.79], "Berlin": [13.40, 52.52],
    "Brussels": [4.35, 50.85], "Bucharest": [26.10, 44.44], "Budapest": [19.04, 47.50],
    "Copenhagen": [12.57, 55.68], "Dublin": [-6.27, 53.35], "Hamburg": [9.99, 53.55],
    "Istanbul": [28.98, 41.02], "Kyiv": [30.52, 50.45], "London": [-0.12, 51.51],
    "Madrid": [-3.70, 40.42], "Milan": [9.19, 45.46], "Moscow": [37.62, 55.75],
    "Munich": [11.58, 48.14], "Paris": [2.35, 48.86], "Prague": [14.42, 50.07],
    "Rome": [12.50, 41.90], "Saint Petersburg": [30.31, 59.94], "Sofia": [23.32, 42.70],
    "Stockholm": [18.06, 60.33], "Vienna": [16.36, 48.21], "Warsaw": [21.02, 52.24]}

```

```

[2]: #Helper code for plotting plans
#First, visualizing the cities.
import csv
with open("european_cities.csv", "r") as f:
    data = list(csv.reader(f, delimiter=';'))
    cities = data[0]

fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(europe_map, extent=[-14.56, 38.43, 37.697 + 0.3, 64.344 + 2.0],
    aspect="auto")

# Map (long, lat) to (x, y) for plotting
for city, location in city_coords.items():
    generations, y = (location[0], location[1])
    plt.plot(generations, y, 'ok', markersize=5)
    plt.text(generations, y, city, fontsize=12)

```

```

[3]: #A method you can use to plot your plan on the map.
def plot_plan(city_order, show=True):
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.imshow(europe_map, extent=[-14.56, 38.43, 37.697 + 0.3, 64.344 + 2.0],
    aspect="auto")

    # Map (long, lat) to (x, y) for plotting
    for index in range(len(city_order) - 1):
        current_city_coords = city_coords[city_order[index]]
        next_city_coords = city_coords[city_order[index+1]]
        x, y = current_city_coords[0], current_city_coords[1]
        #Plotting a line to the next city

```

```

    next_x, next_y = next_city_coords[0], next_city_coords[1]
    plt.plot([x, next_x], [y, next_y])

    plt.plot(x, y, 'ok', markersize=5)
    plt.text(x, y, index, fontsize=12)
    #Finally, plotting from last to first city
    first_city_coords = city_coords[city_order[0]]
    first_x, first_y = first_city_coords[0], first_city_coords[1]
    plt.plot([next_x, first_x], [next_y, first_y])
    #Plotting a marker and index for the final city
    plt.plot(next_x, next_y, 'ok', markersize=5)
    plt.text(next_x, next_y, index+1, fontsize=12)

    if show:
        plt.show()

    else:
        return fig, ax

```

```

[4]: #Example usage of the plotting-method.
plan = list(city_coords.keys()) # Gives us the cities in alphabetic order
print(plan)
plot_plan(plan)

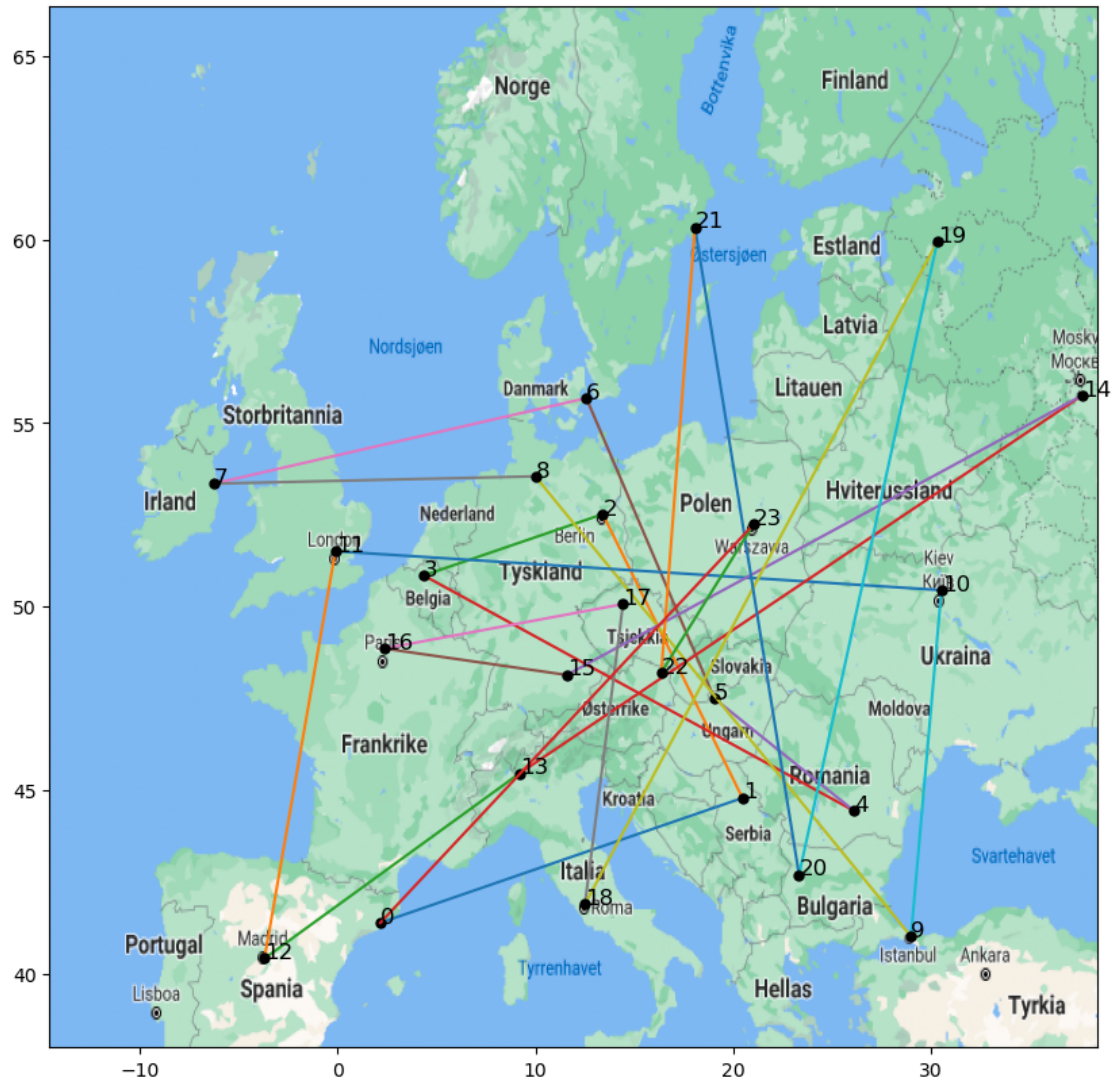
```

```

['Barcelona', 'Belgrade', 'Berlin', 'Brussels', 'Bucharest', 'Budapest',
'Copenhagen', 'Dublin', 'Hamburg', 'Istanbul', 'Kyiv', 'London', 'Madrid',
'Milan', 'Moscow', 'Munich', 'Paris', 'Prague', 'Rome', 'Saint Petersburg',
'Sofia', 'Stockholm', 'Vienna', 'Warsaw']

```





1.2 Imports

```
[ ]: import random
import numpy as np
from time import time
from tqdm import tqdm
from profile import run
from random import sample
import matplotlib.pyplot as plt
from itertools import permutations
from scipy.optimize import curve_fit
from typing import Iterable, Callable
from dataclasses import dataclass, field
```

```
plt.rcParams.update({'font.size': 14})
```

1.3 Shared Functions, Classes and Variables

```
[ ]: def city_dist(city1: str, city2: str) -> float:
    """
    Calculate the distance between two cities.

    Parameters
    -----
    city1 : str
        The name of the first city.
    city2 : str
        The name of the second city.

    Returns
    -----
    float
        The distance between the two cities.

    Raises
    -----
    AssertionError
        If either city1 or city2 is not in the list of cities.
    """

    assert city1 in cities, f"City {city1} not in list of cities"
    assert city2 in cities, f"City {city2} not in list of cities"

    col = data[0].index(city1)
    row = data[0].index(city2) + 1 # +1 because the first row is the list of
    ↪ cities

    return float(data[row][col])

dist_dict = {}
for city in cities:
    dist_dict[city] = {}
    for city2 in cities:
        dist_dict[city][city2] = city_dist(city, city2)
```

```
[7]: def total_distance(cities: list[str]) -> float:
    """
    Calculate the total distance of a path through a list of cities.
    The function computes the total distance by summing up the distances
```

between consecutive cities in the list, including the distance from the last city back to the first city.

Parameters

cities : list of str

A list of city names representing the path.

Returns

float

The total distance of the path through the cities.

"""

```
n = len(cities)
# Summing up the distances between consecutive cities
s = dist_dict[cities[0]][cities[-1]]
s += sum(dist_dict[cities[i]][cities[i+1]] for i in range(n-1))
return s
```

1.4 Exhaustive Search

First, try to solve the problem by inspecting every possible tour. Start by writing a program to find the shortest tour among a subset of the cities (say, **6** of them). Measure the amount of time your program takes. Incrementally add more cities and observe how the time increases. Plot the shortest tours you found using the `plot_plan` method above, for 6 and 10 cities.

Note: To get distances between cities, use the dictionary `data` created by reading the file `europencities.csv`. *Do not* calculate distances based on the coordinates. The actual distances do not only depend on the differences in the coordinates, but also of the curvature of the earth. The distances available in `data` are corrected for this, and contain the actual true distances.

1.4.1 Functions

```
[ ]: # Implement the algorithm here
def exhaustive_search(cities: list[str]) -> tuple[list[str], float]:
    """
    Finds the shortest path that visits all cities and returns to the starting_
    ↪city.
    Parameters
    -----
    cities : list of str
        A list of city names to visit.
    Returns
    -----
    tuple of (list of str, float)
        A tuple containing the shortest path (as a list of city names) and the_
    ↪total distance of that path.
```



```

"""

# Use normal order as first guess
shortest_path = cities.copy()
shortest_distance = total_distance(shortest_path)

# Loop through all permutations of the cities
for perm in permutations(cities):
    perm_distance = total_distance(perm)
    if perm_distance < shortest_distance:
        shortest_distance = perm_distance
        shortest_path = perm

return shortest_path, shortest_distance

```

1.4.2 Solutions

First 6 cities

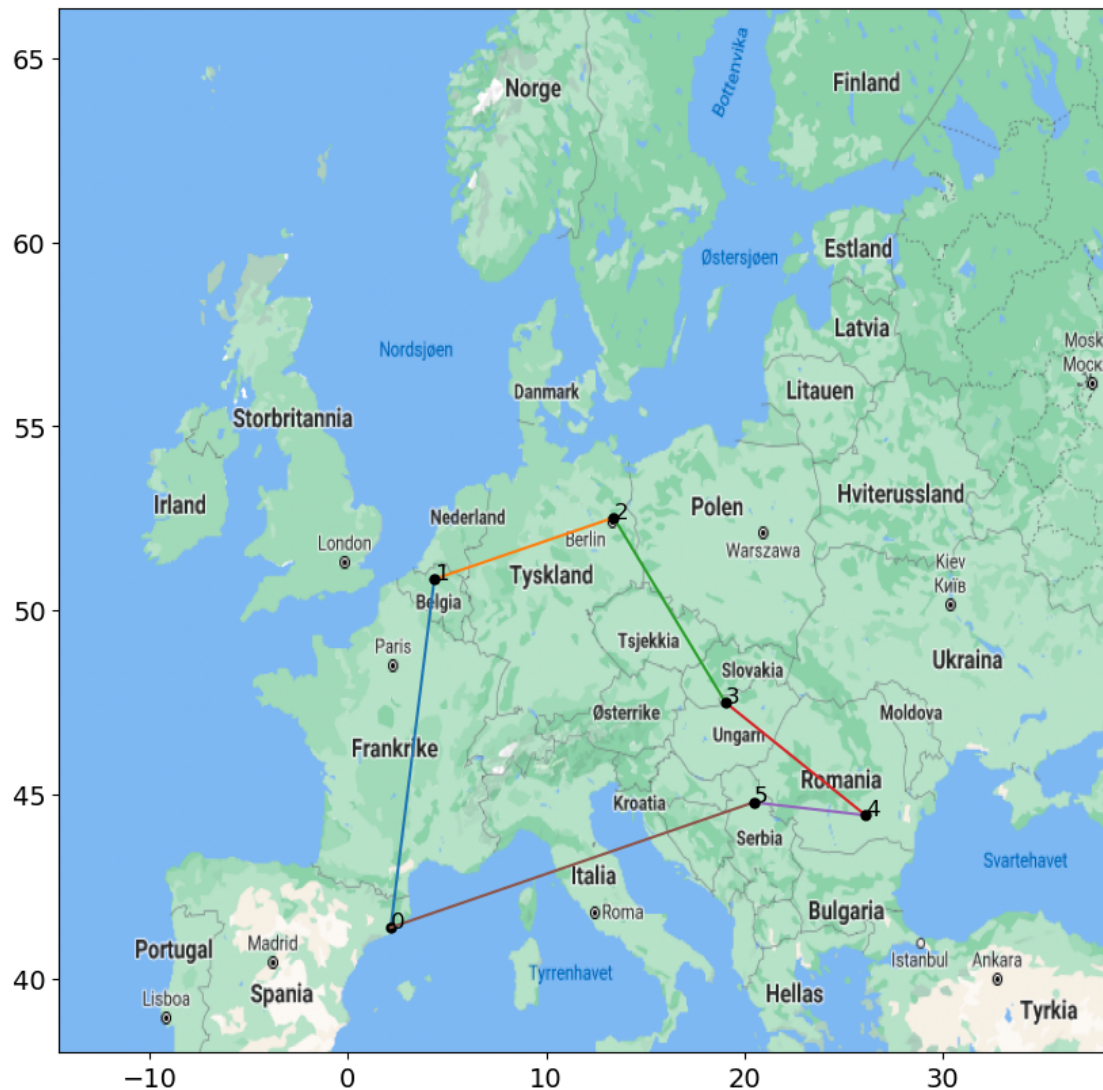
```

[ ]: cities = data[0][:6]
start = time()
path, dist = exhaustive_search(cities)
end = time()
print(f"Shortest path: {' -> '.join(path)}")
print(f"Shortest distance: {dist: .2e} km")
print(f"Execution time: {end-start:.2e} seconds")

plot_plan(path)

```

Shortest path: Barcelona -> Brussels -> Berlin -> Budapest -> Bucharest -> Belgrade
Shortest distance: 5.02e+03 km
Execution time: 3.71e-03 seconds



First 10 cities What is the shortest tour (i.e., the actual sequence of cities, and its length) among the first 10 cities (that is, the cities starting with B,C,D,H and I)? How long did your program take to find it? Calculate an approximation of how long it would take to perform exhaustive search on all 24 cities?

```
[10]: # Answer
cities = data[0][:10]
start = time()
path, dist = exhaustive_search(cities)
end = time()

print(f"Shortest path: {' -> '.join(path)}")
print(f"Shortest distance: {dist: .2e} km")
```

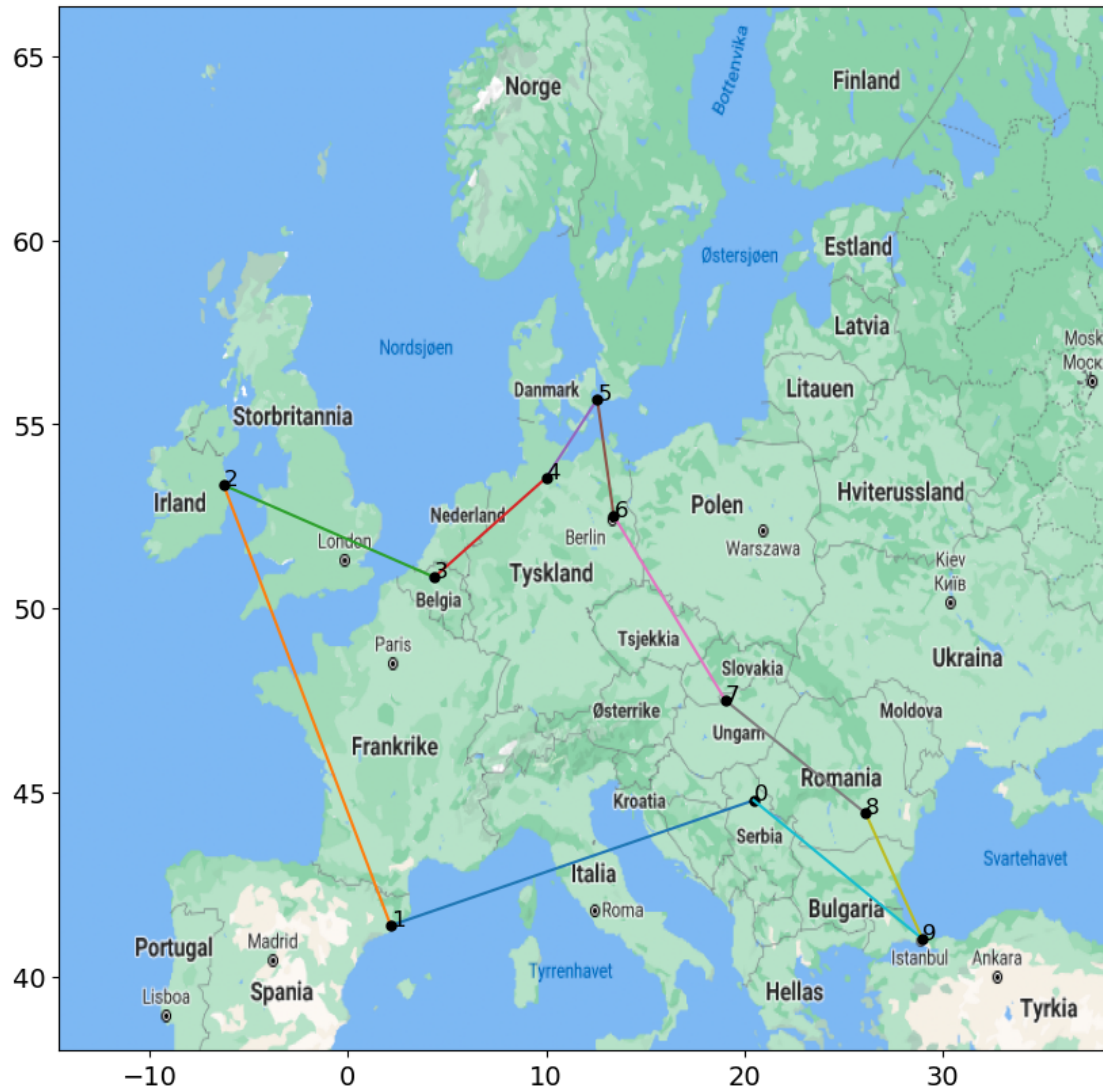
```
print(f"Execution time: {end-start:.2e} seconds")
```

```
plot_plan(path)
plt.show()
```

Shortest path: Belgrade -> Barcelona -> Dublin -> Brussels -> Hamburg -> Copenhagen -> Berlin -> Budapest -> Bucharest -> Istanbul

Shortest distance: 7.49e+03 km

Execution time: 4.50e+00 seconds



Plotting time as a function of the number of cities

```
[ ]: times = []
n_min = 6
```

```

n_max = 10

for n in range(n_min, n_max+1):
    cities = data[0][:n]
    start = time()
    _, dist = exhaustive_search(cities)
    end = time()
    times.append(end-start)

```

```

[12]: plt.semilogy(range(n_min, n_max+1), times, 'o--')
plt.xlabel("Number of cities")
plt.ylabel("Execution time (logscale) [s]")
plt.title("Execution time for finding the shortest path")
plt.xticks(range(n_min, n_max+1))
plt.grid()
plt.show()

```



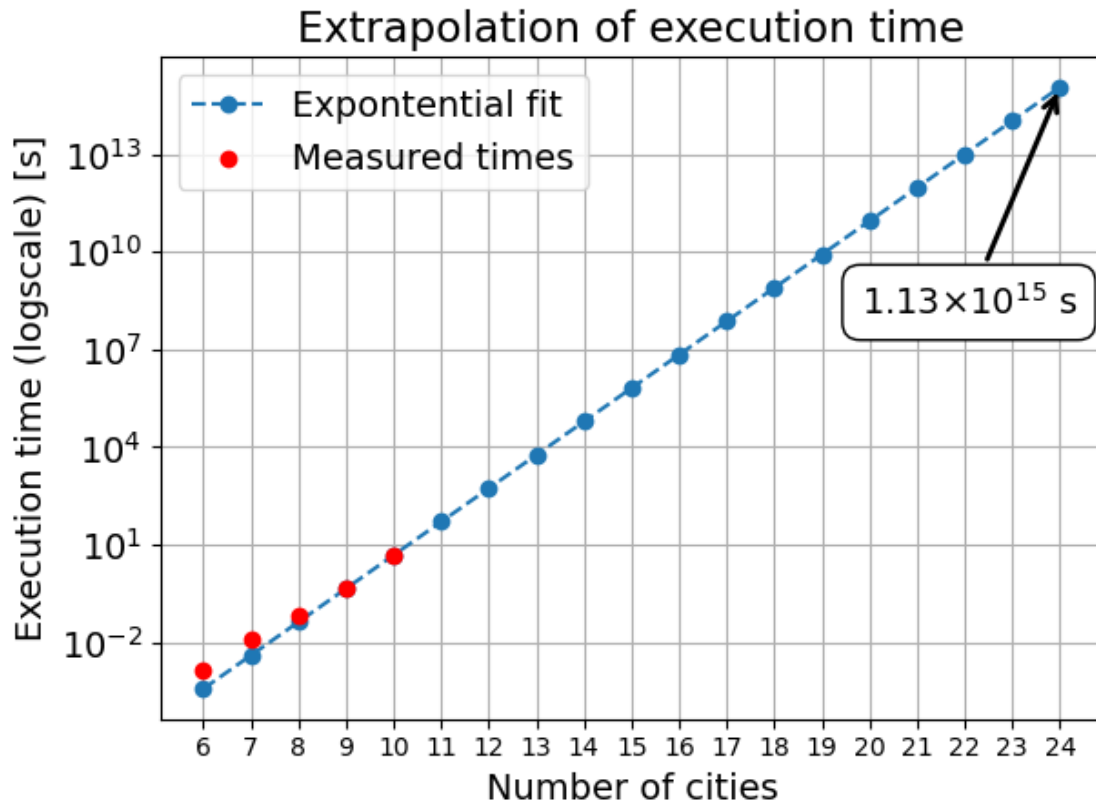
Extrapolation

```
[ ]: a, b = curve_fit(lambda x, a, b: a * np.exp(b * x), range(n_min, n_max+1),
    ↪times)[0]

expansion = 25
x_fit = np.arange(n_min, 25)
y_fit = a * np.exp(b * x_fit)
x_orig = range(n_min, n_max+1)
plt.semilogy(x_fit, y_fit, '--o', label=f"Exponential fit")
plt.scatter(x_orig, times, label="Measured times", color="red", zorder=3)

# Arrow to last point
y_last = f'{y_fit[-1]: .2e}'
plt.annotate(text=rf"${y_last[:5]} × 10^{{{y_last[-2:]}}} s",
             xy=(x_fit[-1], y_fit[-1]),
             xytext=(x_fit[-1]-4.2, y_fit[-1]/10_000_000),
             arrowprops=dict(arrowstyle="→", lw=2),
             bbox=dict(facecolor='white',
                       edgecolor='black',
                       boxstyle='round,pad=0.5'))

plt.xlabel("Number of cities")
plt.ylabel("Execution time (logscale) [s]")
plt.title("Extrapolation of execution time")
plt.xticks(range(n_min, expansion), size=10)
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



Conclusion As seen in the plot above, the execution time (assuming an exponential growth) leads to a runtime of $\approx 10^{14}$ seconds or $\approx 10^7$ years.

1.5 Hill Climbing

Then, write a simple hill climber to solve the TSP. - How well does the hill climber perform, compared to the result from the exhaustive search for the first **10 cities**? - Since you are dealing with a stochastic algorithm, you should run the algorithm several times to measure its performance. - Report the length of the tour of the best, worst and mean of 20 runs (with random starting tours), as well as the standard deviation of the runs, both with the **10 first cities**, and with all **24 cities**. Plot one of the the plans from the 20 runs for both 10 cities and 24 cities (you can use `plot_plan`).

1.5.1 Functions and Classes

```
[ ]: # Implement the algorithm here

class Route():
    def __init__(self, cities: list[str]):
        self._cities: list[str] = cities
        self.potential_fitness = None
        self.calculate_distance()
```

```

def __str__(self) -> str:
    return ' -> '.join(self.cities)

def update_route(self, cities: list[str]) -> None:
    self._cities = cities
    self.calculate_distance()

def calculate_distance(self) -> None:
    self._distance = total_distance(self.cities)
    self._fitness = 1/self.distance

@property
def cities(self) -> list[str]:
    return self._cities

@property
def distance(self) -> float:
    return self._distance

@property
def fitness(self) -> float:
    return self._fitness

```

```

[15]: def single_permutation(cities: list[str]) -> list[str]:
    """
    Creates all single index permutations of the input list

    Parameters
    -----
    cities : list of str
        A list of city names to visit.

    Returns
    -----
    list of list of str
        A list of all single index permutations of the input list
    """

    permutations = []
    for i in range(len(cities)):
        for j in range(i+1, len(cities)):
            permutation = cities.copy()
            permutation[i], permutation[j] = permutation[j], permutation[i]
            permutations.append(permutation)

    return permutations

```

```
[16]: def hill_climbing(cities: list[str]) -> Route:
    """
    Find the shortest path that visits all cities using the hill climbing
    ↪algorithm.

    Parameters
    -----
    cities : list of str
        A list of city names to visit.
    iterations : int
        The number of iterations to perform.

    Returns
    -----
    route : Route
        The shortest route found by the algorithm.
    """

    shortest_route = Route(cities)

    improved = True
    # As long as there is a better path, keep iterating
    while improved:
        permutations = single_permutation(shortest_route.cities)
        for perm in permutations:
            perm_distance = total_distance(perm)
            # Stop at the first improvement and start over
            if perm_distance < shortest_route.distance:
                shortest_route = Route(perm)
                break

        improved = False

    return shortest_route
```

```
[17]: def benchmark_hillclimb(cities: list[str], func_kwargs: dict = dict(),
    ↪iterations: int = 20, plot: bool = False, do_exhaustive_search: bool =
    ↪False) -> None:
    """
    Benchmark the execution time of the hill climb function. Can compare to the
    ↪exhaustive_search function. Prints the results.

    Parameters
    -----
    cities : list of str
        A list of city names to visit.
    func_kwargs : dict
```


Additional keyword arguments to pass to the function
iterations : int
The number of iterations to perform.
plot : bool
If True, the best and worst path found by the function is plotted.
do_exhaustive_search : bool
If True, the exhaustive_search function is also executed and compared_
to the func.

Returns

None

'''

```

if do_exhaustive_search:
    start = time()
    path_es, dist_es = exhaustive_search(cities)
    end = time()
    time_es = end - start

    title = f"Exhaustive Search (n = {len(cities)})"
    print(title)
    print("-"*len(title))
    print(f"Shortest path: {' -> '.join(path_es)}")
    print(f"Shortest distance: {dist_es:.1e} km")
    print(f"Execution time: {time_es:.1e} seconds")
    print()

times = []
paths = []
distances = []

for _ in range(iterations):
    random.shuffle(cities)

    start = time()
    route = hill_climbing(cities, **func_kwargs)
    end = time()

    path = route.cities
    dist = route.distance

    times.append(end-start)
    paths.append(path)
    distances.append(dist)

std_time, mean_time = np.std(times), np.mean(times)
worst_time = max(times)

```

```

best_time = min(times)

std_dist, mean_dist = np.std(distances), np.mean(distances)
worst_dist = max(distances)
best_dist = min(distances)
best_dist_idx = distances.index(best_dist)
worst_dist_idx = distances.index(worst_dist)

title = f"Hill Climbing (n = {len(cities)})"
print(title)
print("-"*len(title))
print(f"Shortest path: {' -> '.join(paths[best_dist_idx])}")
print(f"Longest path: {' -> '.join(paths[worst_dist_idx])}")
print(f"Shortest distance: {best_dist:.1e} km")
print(f"Longest distance: {worst_dist:.1e} km")
print(f"Mean distance: {mean_dist:.1e} ± {std_dist:.1e} km")
print()

print(f"Execution time: {mean_time:.1e} ± {std_time:.1e} seconds")
print(f"Best execution time: {best_time:.1e} seconds")
print(f"Worst execution time: {worst_time:.1e} seconds")
print()

if do_exhaustive_search:
    print(f"Mean runtime improvement: {time_es/mean_time:.1e}x")
    print(f"Best case runtime improvement: {time_es/best_time:.1e}x")
    print(f"Worst case runtime improvement: {time_es/worst_time:.1e}x")
    print()

    print(f"Mean relative error: {abs(mean_dist-dist_es)/dist_es:2.2%}")
    print(f"Best relative error: {abs(best_dist-dist_es)/dist_es:2.2%}")
    print(f"Worst relative error: {abs(worst_dist-dist_es)/dist_es:2.2%}")

if plot:
    plot_plan(paths[best_dist_idx], show=False)
    plt.title(f"Best path found by Hill Climb (n = {len(cities)})")
    plt.show()

    plot_plan(paths[worst_dist_idx], show=False)
    plt.title(f"Worst path found by Hill Climb (n = {len(cities)})")
    plt.show()

```

1.5.2 Solutions

```
[ ]: cities = data[0][:10]

benchmark_hillclimb(cities, iterations=20, plot=True, do_exhaustive_search=True)
```

Exhaustive Search (n = 10)

Shortest path: Belgrade -> Barcelona -> Dublin -> Brussels -> Hamburg ->

Copenhagen -> Berlin -> Budapest -> Bucharest -> Istanbul

Shortest distance: 7.5e+03 km

Execution time: 4.7e+00 seconds

Hill Climbing (n = 10)

Shortest path: Copenhagen -> Bucharest -> Istanbul -> Belgrade -> Budapest ->

Hamburg -> Barcelona -> Dublin -> Brussels -> Berlin

Longest path: Hamburg -> Budapest -> Barcelona -> Bucharest -> Brussels ->

Istanbul -> Dublin -> Belgrade -> Copenhagen -> Berlin

Shortest distance: 8.8e+03 km

Longest distance: 1.5e+04 km

Mean distance: 1.2e+04 ± 1.8e+03 km

Execution time: 1.6e-05 ± 7.0e-06 seconds

Best execution time: 1.1e-05 seconds

Worst execution time: 4.2e-05 seconds

Mean runtime improvement: 3.0e+05x

Best case runtime improvement: 4.1e+05x

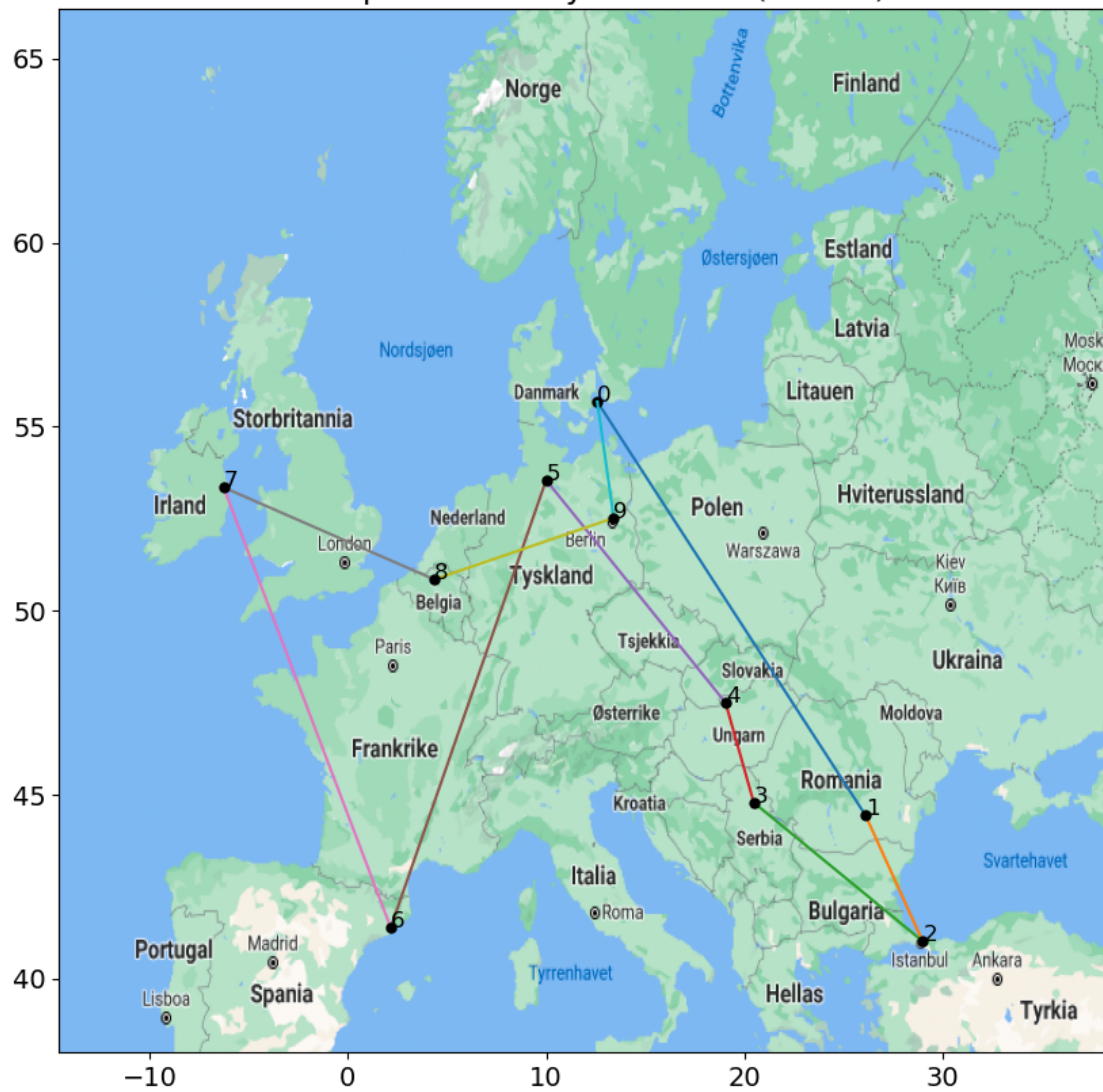
Worst case runtime improvement: 1.1e+05x

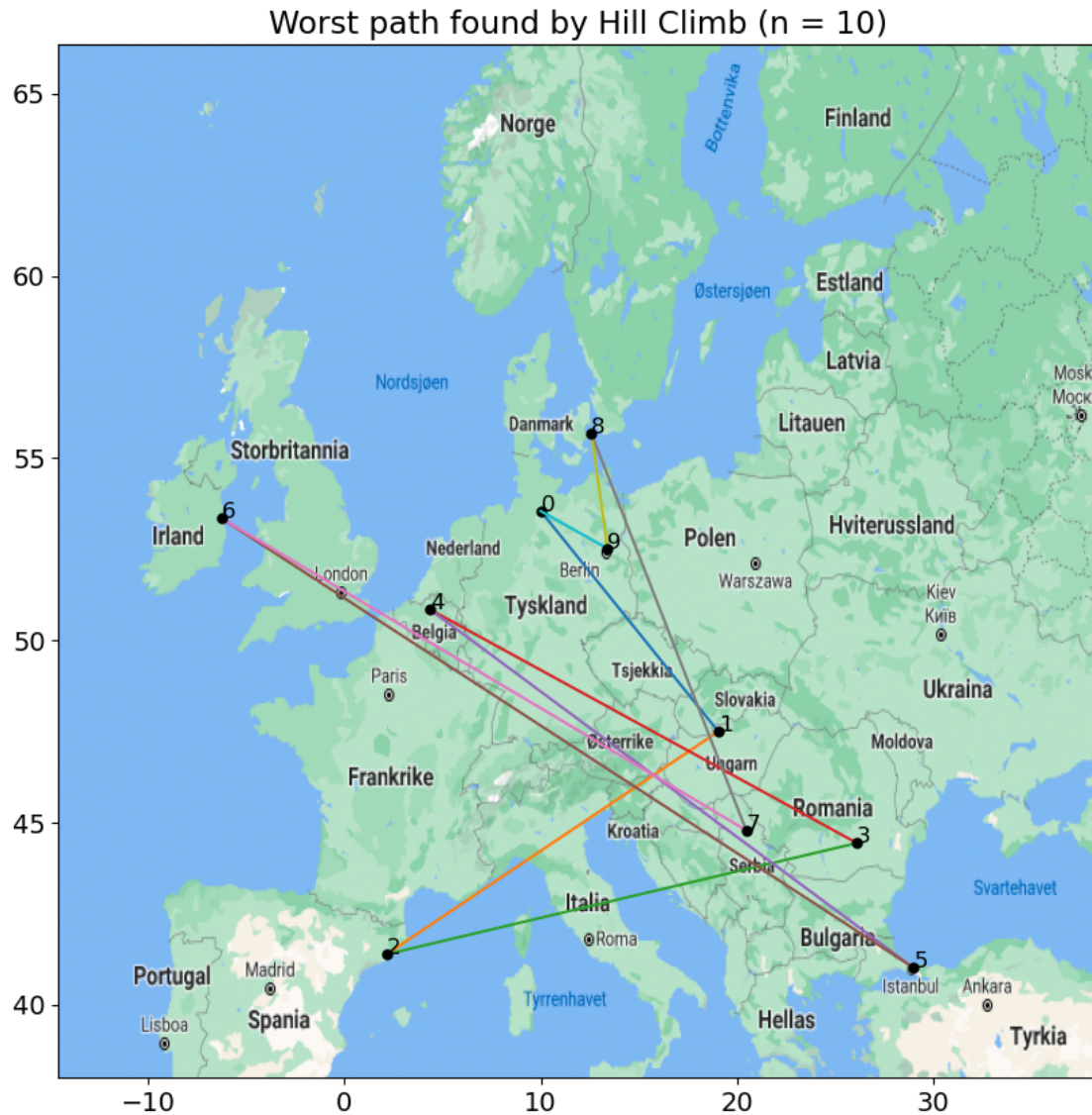
Mean relative error: 65.32%

Best relative error: 17.43%

Worst relative error: 105.37%

Best path found by Hill Climb (n = 10)





Benchmarking first 10 cities

```
[ ]: cities = data[0][:10]

benchmark_hillclimb(cities, iterations=20, plot=True,
    ↪do_exhaustive_search=False)
```

Hill Climbing (n = 10)

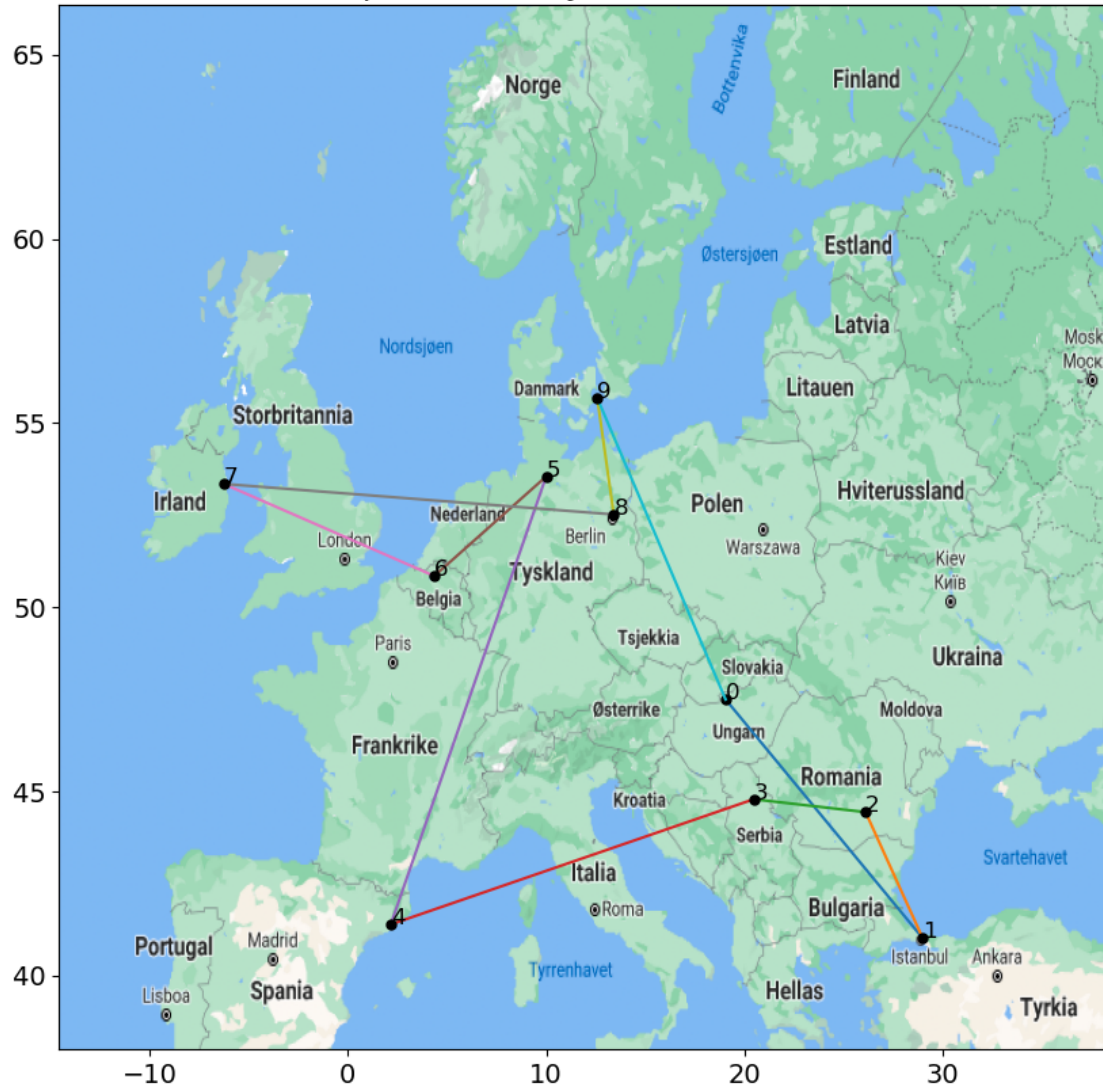
Shortest path: Budapest -> Istanbul -> Bucharest -> Belgrade -> Barcelona -> Hamburg -> Brussels -> Dublin -> Berlin -> Copenhagen

Longest path: Budapest -> Copenhagen -> Dublin -> Bucharest -> Barcelona -> Berlin -> Belgrade -> Brussels -> Istanbul -> Hamburg

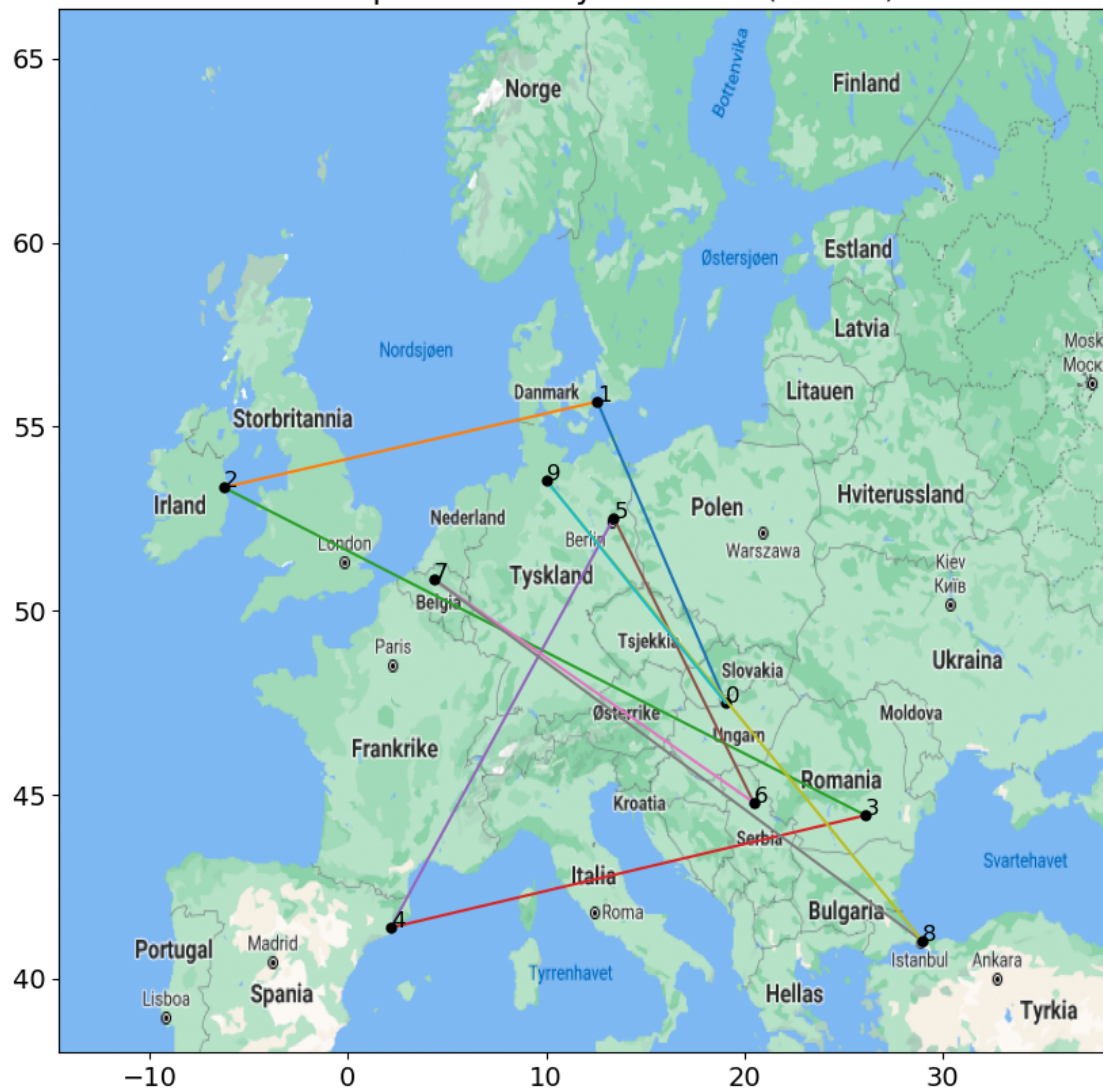
Shortest distance: 8.9×10^3 km
Longest distance: 1.6×10^4 km
Mean distance: $1.2 \times 10^4 \pm 1.8 \times 10^3$ km

Execution time: $6.6 \times 10^{-5} \pm 3.5 \times 10^{-5}$ seconds
Best execution time: 4.6×10^{-5} seconds
Worst execution time: 2.1×10^{-4} seconds

Best path found by Hill Climb (n = 10)



Worst path found by Hill Climb (n = 10)



Benchmarking all 24 cities

```
[ ]: cities = data[0].copy()
benchmark_hillclimb(cities, iterations=20, plot=True,
do_exhaustive_search=False)
```

Hill Climbing (n = 24)

Shortest path: Paris -> Hamburg -> Dublin -> Saint Petersburg -> Copenhagen -> Moscow -> Stockholm -> Berlin -> Brussels -> Prague -> Kyiv -> Vienna -> Bucharest -> Warsaw -> Budapest -> London -> Belgrade -> Sofia -> Madrid -> Barcelona -> Istanbul -> Rome -> Milan -> Munich

Longest path: Barcelona -> Budapest -> Vienna -> Dublin -> Istanbul -> Sofia ->

London -> Saint Petersburg -> Rome -> Madrid -> Berlin -> Paris -> Stockholm ->
Munich -> Hamburg -> Belgrade -> Milan -> Kyiv -> Brussels -> Moscow ->
Bucharest -> Warsaw -> Copenhagen -> Prague

Shortest distance: $2.6e+04$ km

Longest distance: $3.4e+04$ km

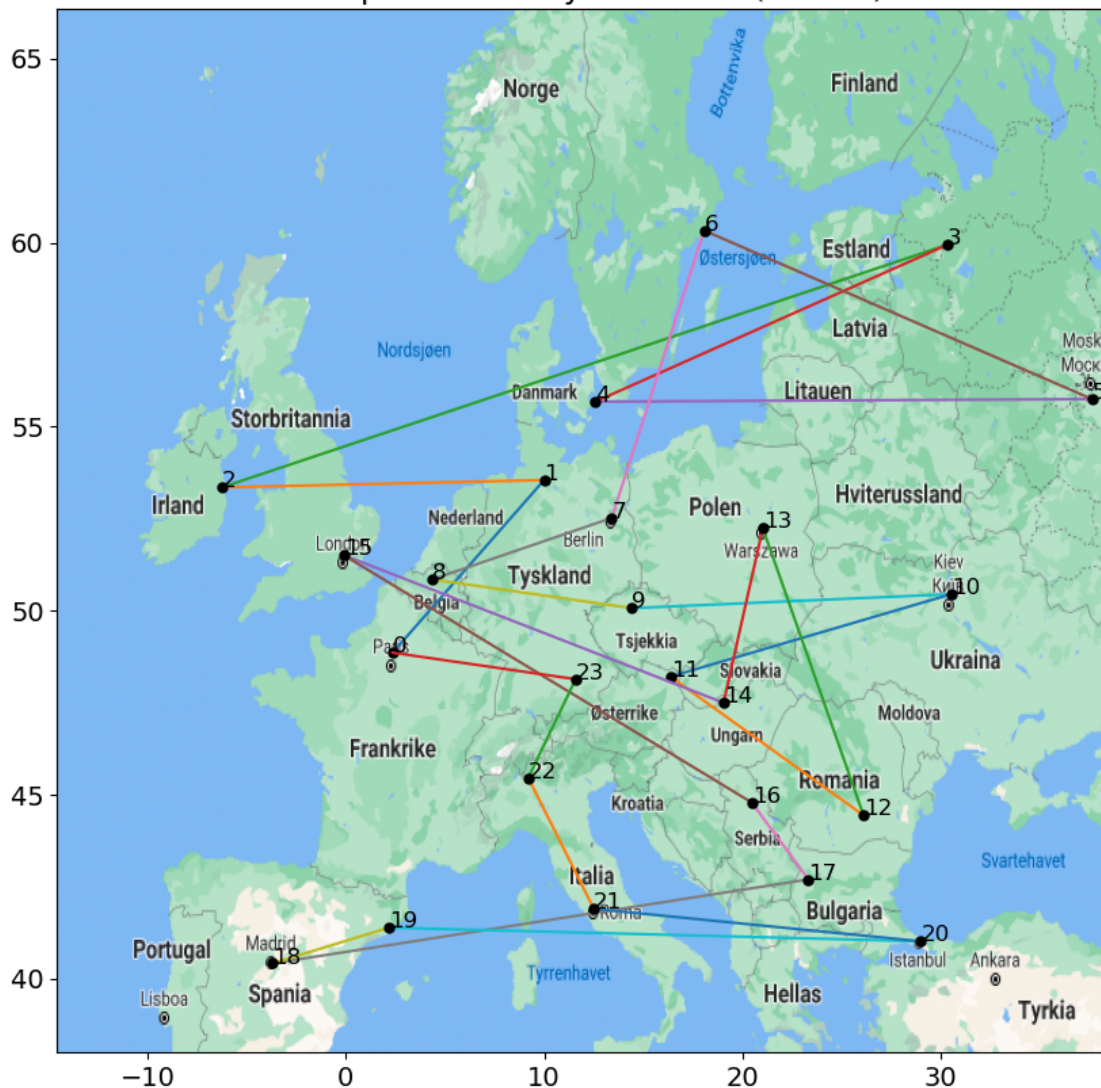
Mean distance: $3.1e+04 \pm 1.9e+03$ km

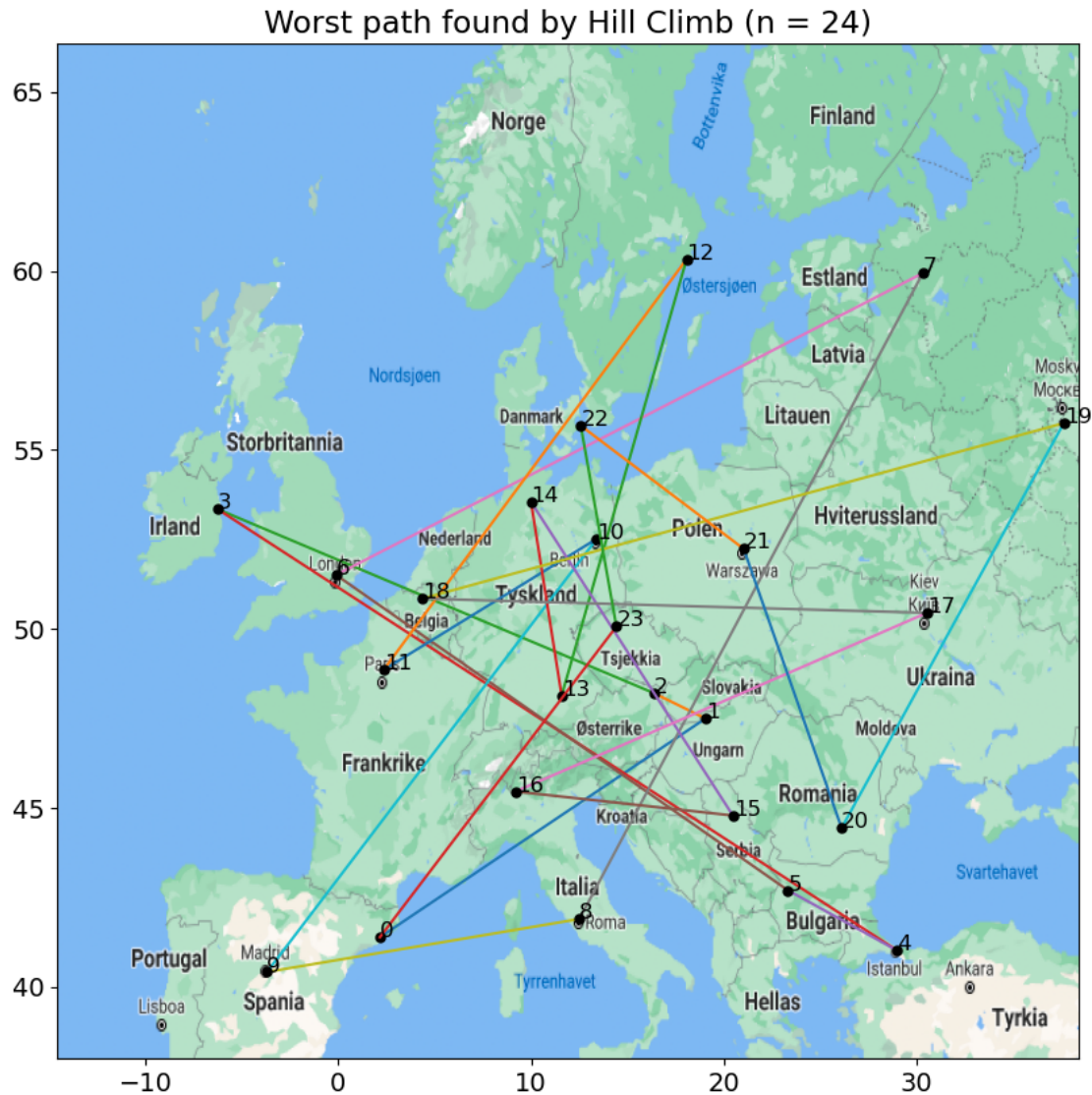
Execution time: $2.6e-04 \pm 1.7e-04$ seconds

Best execution time: $1.7e-04$ seconds

Worst execution time: $9.9e-04$ seconds

Best path found by Hill Climb (n = 24)





1.6 Genetic Algorithm

Next, write a genetic algorithm (GA) to solve the problem. Choose mutation and crossover operators that are appropriate for the problem (see chapter 4.5 of the Eiben and Smith textbook). Choose three different values for the population size. Define and tune other parameters yourself and make assumptions as necessary (and report them, of course).

For all three variants: As with the hill climber, report best, worst, mean and standard deviation of tour length out of 20 runs of the algorithm (of the best individual of last generation). Also, find and plot the average fitness of the best fit individual in each generation (average across runs), and include a figure with all three curves in the same plot in the report. This means that the x-axis should be the generations over time and the y-axis should be the average (over the 20-runs) fitness of the best gene in that generation. Conclude which is best in terms of tour length and number of

generations of evolution time.

Finally, plot an example optimized tour (the best of the final generation) for the three different population sizes, using the `plot_plan` method.

1.6.1 Functions and Classes

Crossover

```
[ ]: def select_segment(n: int) -> slice:
    """
    Selects a random segment of a list of length n.

    Parameters
    -----
    n : int
        The length of the list.

    Returns
    -----
    slice
        A slice object representing the segment.
    """
    idx1 = np.random.randint(0, n)
    idx2 = np.random.randint(0, n)

    # Make sure idx1 != idx2
    while idx1 == idx2:
        idx2 = np.random.randint(0, n)

    # Make sure idx1 always refers to the smaller index
    if idx1 > idx2:
        idx2, idx1 = idx1, idx2

    segment = slice(idx1, idx2+1)
    return segment

def pmx(p1: Iterable, p2: Iterable, segment: slice = None) -> tuple[np.ndarray,
↪ np.ndarray]:
    """
    Partially Mapped Crossover

    Parameters
    -----
    p1 : Iterable
        The first parent.
    p2 : Iterable
```

```

    The second parent.
    segment : slice (optional)
    The segment to be swapped between the parents. If None, a random
    ↪segment is selected.

Returns
-----
tuple of np.ndarray
    The two children created by the Partially Mapped Crossover.
    ...
def procedure(p1: np.ndarray, p2: np.ndarray, segment: slice):
    # Step 1
    child = np.zeros_like(p1)
    child[segment] = p1[segment]

    # Step 2
    for gene1 in p1[segment]:
        gene1_idx = np.where(p1 == gene1)
        gene2 = p2[gene1_idx]

        if gene2 in child:
            continue

        gene1_in_p2_idx = np.where(p2 == gene1)
        while gene1 in p2[segment]:
            gene1 = p1[gene1_in_p2_idx]
            gene1_in_p2_idx = np.where(p2 == gene1)

        child[gene1_in_p2_idx] = gene2

    # Step 3
    for idx, gene in enumerate(child):
        # Checks for empty spots. Works for both strings (gene == "") and
        ↪numbers (gene == 0)
        if not gene:
            child[idx] = p2[idx]

    return child

# Preprocessing
## Select segment if not given. Implemented for testing purposes.
if segment is None:
    segment = select_segment(len(p1))

p1, p2 = np.array(p1), np.array(p2)
child1 = procedure(p1, p2, segment)
child2 = procedure(p2, p1, segment)

```

```

    return child1, child2

def order_crossover(p1: Iterable, p2: Iterable, segment: slice = None) -> ␣
    tuple[np.ndarray, np.ndarray]:
    """
    Order Crossover

    Parameters
    -----
    p1 : Iterable
        The first parent.
    p2 : Iterable
        The second parent.
    segment : slice (optional)
        The segment to be swapped between the parents. If None, a random ␣
    segment is selected.

    Returns
    -----
    tuple of np.ndarray
        The two children created by the Order Crossover.
    """
    def procedure(p1: np.ndarray, p2: np.ndarray, segment: slice):
        n = len(p1)
        # Step 1
        idx1 = segment.start
        idx2 = segment.stop - 1

        child = np.zeros_like(p1) # Works for any dtype
        child[segment] = p1[segment]

        # Step 2
        next_available_index = (idx2 + 1) % n
        for gene in np.roll(p2, -idx2 - 1): # Aligns the gene after idx2 to the ␣
        beginning for easier iteration
            if gene not in child:
                child[next_available_index % n] = gene
                next_available_index += 1

        return child

    # Preprocessing
    ## Select segment if not given. Implemented for testing purposes.
    if segment is None:
        segment = select_segment(len(p1))

```

```

p1, p2 = np.array(p1), np.array(p2)
child1 = procedure(p1, p2, segment)
child2 = procedure(p2, p1, segment)

return child1, child2

def cycle_crossover(p1: Iterable, p2: Iterable) -> tuple[np.ndarray, np.
    ndarray]:
    """
    Cycle Crossover

    Parameters
    -----
    p1 : Iterable
        The first parent.
    p2 : Iterable
        The second parent.

    Returns
    -----
    tuple of np.ndarray
        The two children created by the Cycle Crossover.
    """
    p1, p2 = np.array(p1), np.array(p2)

    # Step 1: Find Cycles
    cycles = []
    for i in range(len(p1)):
        allele = p1[i]
        allele_adj = p2[i]
        idx = np.where(p1 == allele_adj)[0][0]

        # Check for element already in a cycle
        visited = False
        for cycle in cycles:
            if idx in cycle:
                visited = True
                break

        if visited:
            continue

        # Find cycle
        ## First element
        cycle = []

```

```

        cycle.append(idx)

        first = allele
        allele = p1[idx]
        allele_adj = p2[idx]
        ## The rest of the elements
        while first != allele:
            idx = np.where(p1 == allele_adj)[0][0]
            cycle.append(idx)
            allele = p1[idx]
            allele_adj = p2[idx]

        cycles.append(cycle)

    child1 = np.zeros_like(p1)
    child2 = np.zeros_like(p2)

    # Step 2: Alternating parents
    for cycle in cycles:
        for idx in cycle:
            child1[idx] = p1[idx]
            child2[idx] = p2[idx]

    p1, p2 = p2, p1

    return child1, child2

```

Testing crossover functions

```

[ ]: # Test case (lecture 3, slide 56)
p1_int = (1, 2, 3, 4, 5, 6, 7, 8, 9)
p2_int = (9, 3, 7, 8, 2, 6, 5, 1, 4)
child1_correct_int = [9, 3, 2, 4, 5, 6, 7, 1, 8]
child2_correct_int = [1, 7, 3, 8, 2, 6, 5, 4, 9]

p1_str = ('1', '2', '3', '4', '5', '6', '7', '8', '9')
p2_str = ('9', '3', '7', '8', '2', '6', '5', '1', '4')
child1_correct_str = ['9', '3', '2', '4', '5', '6', '7', '1', '8']
child2_correct_str = ['1', '7', '3', '8', '2', '6', '5', '4', '9']

child1, child2 = pmx(p1_int, p2_int, segment=slice(3,7))
assert child1.tolist() == child1_correct_int, f"Expected {child1_correct_int},  

    ↳ got {child1}"
assert child2.tolist() == child2_correct_int, f"Expected {child2_correct_int},  

    ↳ got {child2}"

child1, child2 = pmx(p1_str, p2_str, segment=slice(3,7))

```

```

assert child1.tolist() == child1_correct_str, f"Expected {child1_correct_str},  

↳got {child1}"
assert child2.tolist() == child2_correct_str, f"Expected {child2_correct_str},  

↳got {child2}"
#####

# Test case (lecture 3, slide 61)
p1_int = (1, 2, 3, 4, 5, 6, 7, 8, 9)
p2_int = (9, 3, 7, 8, 2, 6, 5, 1, 4)
child1_correct_int = [3, 8, 2, 4, 5, 6, 7, 1, 9]
child2_correct_int = [3, 4, 7, 8, 2, 6, 5, 9, 1]

p1_str = ('1', '2', '3', '4', '5', '6', '7', '8', '9')
p2_str = ('9', '3', '7', '8', '2', '6', '5', '1', '4')
child1_correct_str = ['3', '8', '2', '4', '5', '6', '7', '1', '9']
child2_correct_str = ['3', '4', '7', '8', '2', '6', '5', '9', '1']

child1, child2 = order_crossover(p1_int, p2_int, segment=slice(3,7))
assert child1.tolist() == child1_correct_int, f"Expected {child1_correct_int},  

↳got {child1}"
assert child2.tolist() == child2_correct_int, f"Expected {child2_correct_int},  

↳got {child2}"

child1, child2 = order_crossover(p1_str, p2_str, segment=slice(3,7))
assert child1.tolist() == child1_correct_str, f"Expected {child1_correct_str},  

↳got {child1}"
assert child2.tolist() == child2_correct_str, f"Expected {child2_correct_str},  

↳got {child2}"
#####

# Test case (lecture 3, slide 63)
p1_int = (1, 2, 3, 4, 5, 6, 7, 8, 9)
p2_int = (9, 3, 7, 8, 2, 6, 5, 1, 4)
child1_correct_int = [1, 3, 7, 4, 2, 6, 5, 8, 9]
child2_correct_int = [9, 2, 3, 8, 5, 6, 7, 1, 4]

p1_str = ('1', '2', '3', '4', '5', '6', '7', '8', '9')
p2_str = ('9', '3', '7', '8', '2', '6', '5', '1', '4')
child1_correct_str = ['1', '3', '7', '4', '2', '6', '5', '8', '9']
child2_correct_str = ['9', '2', '3', '8', '5', '6', '7', '1', '4']

child1, child2 = cycle_crossover(p1_int, p2_int)
assert child1.tolist() == child1_correct_int, f"Expected {child1_correct_int},  

↳got {child1}"
assert child2.tolist() == child2_correct_int, f"Expected {child2_correct_int},  

↳got {child2}"

```



```

child1, child2 = cycle_crossover(p1_str, p2_str)
assert child1.tolist() == child1_correct_str, f"Expected {child1_correct_str},  

↳got {child1}"
assert child2.tolist() == child2_correct_str, f"Expected {child2_correct_str},  

↳got {child2}"

```

Classes and Dataclasses

```

[ ]: @dataclass
class ga_result():
    """
    A dataclass to store the results of a genetic algorithm.

    Attributes
    -----
    best_route : Route
        The best route found by the genetic algorithm.
    mean_dist : float
        The mean distance of all paths.
    std_dist : float
        The standard deviation of the distances.
    exec_time : float
        The execution time of the genetic algorithm.
    best_dist_history : list of float
        The history of the best distance found.
    mean_dist_history : list of float
        The history of the mean distance found.
    """
    best_route: Route = None
    mean_dist: float = None
    std_dist: float = None
    exec_time: float = None
    best_dist_history: list[float] = field(default_factory=list)
    mean_dist_history: list[float] = field(default_factory=list)

@dataclass
class ga_experiment_result():
    """
    A dataclass to store the results of multiple genetic algorithm experiments.

    Attributes
    -----
    distances : np.ndarray
        An array of the best distances found by the genetic algorithm.
    histories : np.ndarray
        An array of the best distance histories.
    routes : list[Route]

```

```

        A list of the best routes found by the genetic algorithm.
    exec_times : np.ndarray
        An array of the execution times of the genetic algorithm.
    """
    distances: np.ndarray
    histories: np.ndarray
    routes: list[Route]
    exec_times: np.ndarray

```

Algorithm Class

```

[ ]: class EGA():
    """
    Purpose
    -----
    EGA (Evolutionary Genetic Algorithm) class for solving the Traveling_
    ↪Salesman Problem (TSP).

    This class implements a genetic algorithm to find an optimal or_
    ↪near-optimal solution to the TSP.
    It includes methods for creating a population of routes, evaluating their_
    ↪fitness, performing
    selection, crossover, and mutation operations, and running the genetic_
    ↪algorithm over multiple
    generations.

    Methods
    -----
    create_population(cities: list[str], population_size: int) -> list[Route]
        Create a population of random routes.
    evaluate_population(cls, population: list[Route]) -> list[Route]
        Evaluate the population based on the fitness of the routes.
    fitness_selection(population: list[Route]) -> list[Route]
        Sort the population based on the fitness of the routes.
    tournament_selection(population: list[Route], n: int) -> list[Route]
        Select the best route in terms of fitness from a random sample of the_
    ↪population.
    inv_mutation(route: Route) -> Route
        Perform an inversion mutation on a route.
    swap_mutation(route: Route) -> Route
        Perform a swap mutation on a route.
    mutation(cls, route: Route) -> Route
        Perform a mutation on a route with a 50% chance of either inversion or_
    ↪swap mutation.
    crossover(p1: Route, p2: Route) -> tuple[Route, Route]
        Perform a crossover on two routes with a 50% chance of either PMX or_
    ↪Order crossover.

```

```

    __call__(cls, cities: list[str], population_size: int, n_generations: int)
    ↪ -> ga_result
        Run the genetic algorithm.

    """

    def __init__(self):
        pass

    @staticmethod
    def create_population(cities: list[str], population_size: int) ->
    ↪ list[Route]:
        """
        Create a population of random routes.

        Parameters
        -----
        cities : list of str
            A list of city names to visit.
        population_size : int
            The size of the population.

        Returns
        -----
        list of Route
            A list of random routes.
        """
        return [Route(np.random.permutation(cities)) for _ in
    ↪ range(population_size)]

    @classmethod
    def evaluate_population(cls, population: list[Route]) -> list[Route]:
        """
        Evaluate the population based on the fitness of the routes.
        Subject to change based on which evolutionary algoirithm is used.
        """
        return cls.fitness_selection(population)

    @staticmethod
    def fitness_selection(population: list[Route]) -> list[Route]:
        """
        Sort the population based on the fitness of the
        Subject to change based on which evolutionary algoirithm is used.
        """
        return sorted(population, key=lambda route: route.fitness, reverse=True)

    @staticmethod

```

```

def tournament_selection(population: list[Route], n: int) -> list[Route]:
    """
    Selects the best route in terms of fitness from a random sample of the
    ↪population.
    Subject to change based on which evolutionary algorithm is used.
    """
    return max(sample(population, n), key=lambda route: route.fitness)

@staticmethod
def inv_mutation(route: Route) -> Route:
    """
    Performs an inversion mutation on a route
    """
    n = len(route.cities)
    s = select_segment(n)
    idx1, idx2 = s.start, s.stop - 1
    route.cities[idx1:idx2+1] = route.cities[idx1:idx2+1][::-1]
    return route

@staticmethod
def swap_mutation(route: Route) -> Route:
    """
    Performs a swap mutation on a route'
    """
    n = len(route.cities)
    idx1, idx2 = np.random.randint(0, n), np.random.randint(0, n)
    route.cities[idx1], route.cities[idx2] = route.cities[idx2], route.
    ↪cities[idx1]
    return route

@classmethod
def mutation(cls, route: Route) -> Route:
    """
    Performs a mutation on a route with a 50% chance of either inversion or
    ↪swap mutation.
    """
    if np.random.rand() < 0.5:
        mut = cls.inv_mutation
    else:
        mut = cls.swap_mutation

    route = mut(route)
    route.calculate_distance()
    return route

@staticmethod
def crossover(p1: Route, p2: Route) -> tuple[Route, Route]:

```

```

'''
    Performs a crossover on two routes with a 50% chance of either PMX or
    ↪Order crossover.
'''
    if np.random.rand() < 0.5:
        cross = pmx
    else:
        cross = order_crossover

    path1, path2 = cross(p1.cities, p2.cities)

    child1 = Route(path1)
    child2 = Route(path2)

    return child1, child2

@classmethod
def __call__(cls, cities: list[str], population_size: int, n_generations:
    ↪int) -> ga_result:
    '''
        Genetic Algorithm

        Parameters
        -----
        cities : list of str
            A list of city names to visit.
        gen_size : int
            The size of each generation.
        gen_num : int
            The number of generations to perform the genetic algorithm.

        Returns
        -----
        result : ga_result
            The result of the genetic algorithm stored in a ga_result dataclass.
    '''
    start = time()

    # Shuffling all members of the population
    population = cls.create_population(cities, population_size)

    # Number of elites to keep
    elitism = int(population_size//2)

    # Dataclass to store the results
    result = ga_result()

```

```

# Main loop
for _ in range(n_generations):
    # Remove the bottom % of the population
    population = cls.evaluate_population(population)[: -elitism]

    # Only let the best reproduce to fill the population back up
    children = []
    for _ in range(0, elitism, 2):
        p1 = cls.tournament_selection(population, elitism)
        p2 = cls.tournament_selection(population, elitism)

        child1, child2 = cls.crossover(p1, p2)

        child1 = cls.mutation(child1)
        child2 = cls.mutation(child2)

        children.extend([child1, child2])

    population.extend(children)

    best = cls.fitness_selection(population)[0]
    # Keep track of the best individual in each generation
    result.best_dist_history.append(best.distance)

    result.best_route = cls.fitness_selection(population)[0]
    result.mean_dist = np.mean([route.distance for route in population])
    result.std_dist = np.std([route.distance for route in population])

    end = time()
    result.exec_time = end - start
    return result

```

```

[25]: def plot_convergence(histories: list[list[float]], population_sizes: list[int],
    ↪ show: bool = True, zoom = False) -> None | plt.Figure:
    """
    Plots the convergence of the genetic algorithm comparing different
    ↪ population sizes.

    Parameters
    -----
    histories : list of list of float
        A list of the best distance histories.
    population_sizes : list of int
        A list of the population sizes used in the genetic algorithm. Assumed
    ↪ to be in ascending order.
    show : bool
        If True, the plot is shown. If False, the figure is returned.

```

```

zoom : bool
    If True, a zoomed in plot of the last 20% of the generations is shown.

Returns
-----
None / plt.Figure
    If `show` is False, the figure is returned.
'''
histories = np.array(histories)/1_000
generations = np.arange(1, histories.shape[-1]+1)
pad = len(str(max(population_sizes)))
for hist, pop_size in zip(histories, population_sizes):
    avg_hist = np.mean(hist, axis=0)
    plt.plot(generations, avg_hist, 'o--', markersize=3,
↳label=f"Population size: {pop_size:>{pad}}")

plt.xlabel("Generation")
plt.ylabel("Distance [miles]")
plt.title("Convergence of Genetic Algorithm")
plt.legend()

if zoom:
    left, bottom, width, height = 0.75, 0.35, 0.25, 0.25
    ax = plt.gca()
    axins = ax.inset_axes([left, bottom, width, height])
    for hist, pop_size in zip(histories, population_sizes):
        avg_hist = np.mean(hist, axis=0)
        axins.plot(generations, avg_hist, 'o--', markersize=3,
↳label=f"Population size: {pop_size}")
        last_10_gens = len(generations)//5
        while last_10_gens > 10:
            last_10_gens = last_10_gens//2

        right_pad = 1
        axins.set_xlim(generations[-last_10_gens:][0], generations[-1] +
↳right_pad)
        axins.set_xticks(generations[-last_10_gens::2])
        axins.set_ylim(12.25, 13.75)
        axins.tick_params(axis='both', which='major', labelsize=8)
        axins.set_title("Last Generations", fontsize=10)
        ax.indicate_inset_zoom(axins)

if show:
    plt.show()

else:

```

```
return plt.gcf()
```

```
[26]: def run_ga_experiment(cities: list[str], population_sizes: list[int],  
    ↪ n_generations: int, genetic_alg: Callable, n_runs: int = 20):  
    distances = []  
    histories = []  
    routes = []  
    exec_times = []  
  
    for pop_size in tqdm(population_sizes):  
        dists = []  
        hists = []  
        route = []  
        exec_time = []  
        for _ in range(n_runs):  
            result = genetic_alg(cities, pop_size, n_generations)  
            dists.append(result.best_route.distance)  
            hists.append(result.best_dist_history)  
            route.append(result.best_route)  
            exec_time.append(result.exec_time)  
  
        distances.append(dists)  
        histories.append(hists)  
        routes.append(route)  
        exec_times.append(exec_time)  
  
    distances = np.array(distances)  
    histories = np.array(histories)  
    exec_times = np.array(exec_times)  
  
    result = ga_experiment_result(distances, histories, routes, exec_times)  
  
    return result
```

```
[27]: def benchmark_evo_alg(experiment_result: ga_experiment_result):  
    routes = experiment_result.routes  
    distances = experiment_result.distances  
    histories = experiment_result.histories  
    exec_times = experiment_result.exec_times  
  
    route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]  
    route_longest = sorted(routes[-1], key=lambda route: route.distance)[-1]  
    distance_mean = np.mean(distances[-1])  
    distance_std = np.std(distances[-1])  
    exec_shortest = sorted(exec_times[-1])[0]  
    exec_longest = sorted(exec_times[-1])[-1]  
    exec_mean = np.mean(exec_times[-1])
```



```

exec_std = np.std(exec_times[-1])

print("Results")
print("-"*len("Results"))

print(f"Shortest path: {route_shortest}")
print()
print(f"Longest path: {route_longest}")
print()

print(f"Shortest distance: {route_shortest.distance:.0f} km")
print(f"Longest distance: {route_longest.distance:.0f} km")
print(f"Mean distance: {distance_mean:.0f} ± {distance_std:.0f} km")
print()

print(f"Shortest execution time: {exec_shortest:.1e} seconds")
print(f"Longest execution time: {exec_longest:.1e} seconds")
print(f"Mean execution time: {exec_mean:.1e} ± {exec_std:.1e} seconds")

```

1.6.2 10 cites with population size 10, 20,..., 90, 100

```

[ ]: cities = data[0][:10]
population_sizes_10 = [i*10 for i in range(1, 11)]
n_generations = 40

experiment_result_10 = run_ga_experiment(cities, population_sizes_10,
↪n_generations, EGA())

```

100%| | 10/10 [00:24<00:00, 2.45s/it]

```

[29]: routes = experiment_result_10.routes
distances = experiment_result_10.distances
histories = experiment_result_10.histories
exec_times = experiment_result_10.exec_times

route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
print(f"Shortest path: {route_shortest}")
print(f"Shortest distance: {route_shortest.distance:.0f} km")

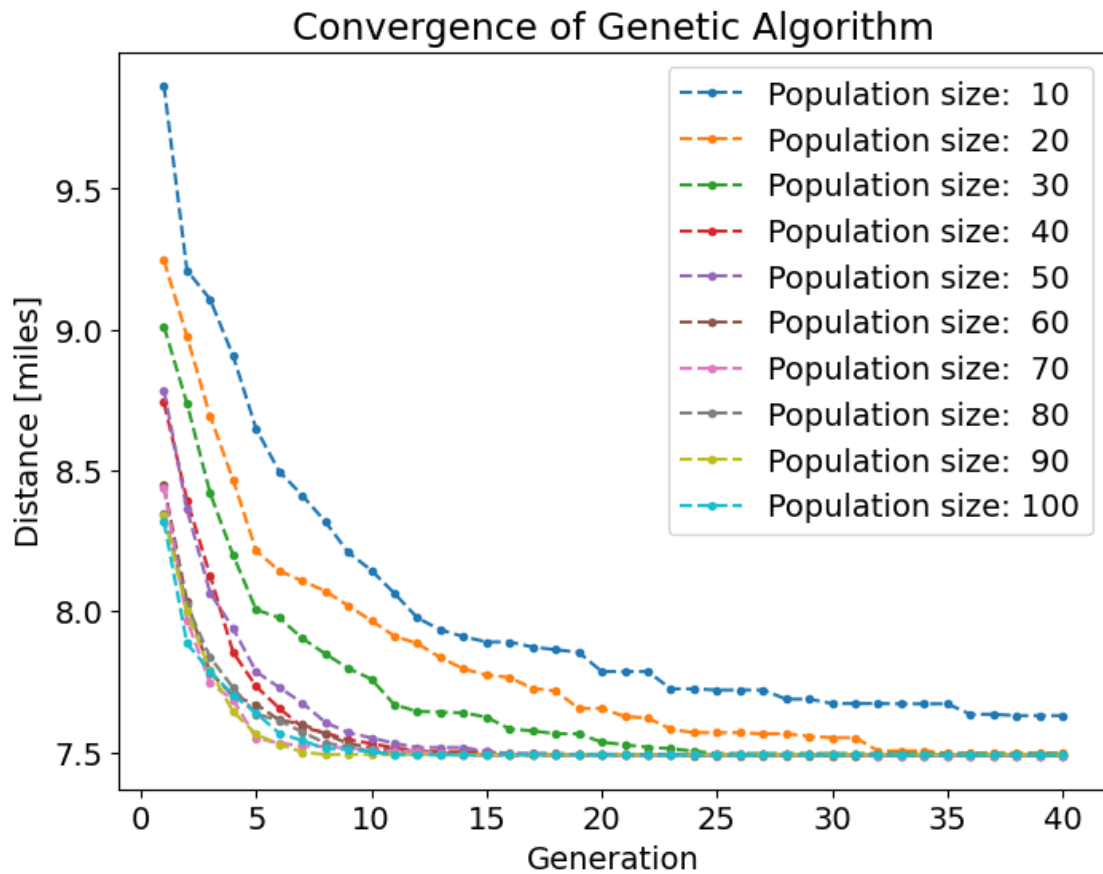
fig = plot_convergence(histories, population_sizes_10, show=False)
fig.set_size_inches(8, 6)
plt.show()

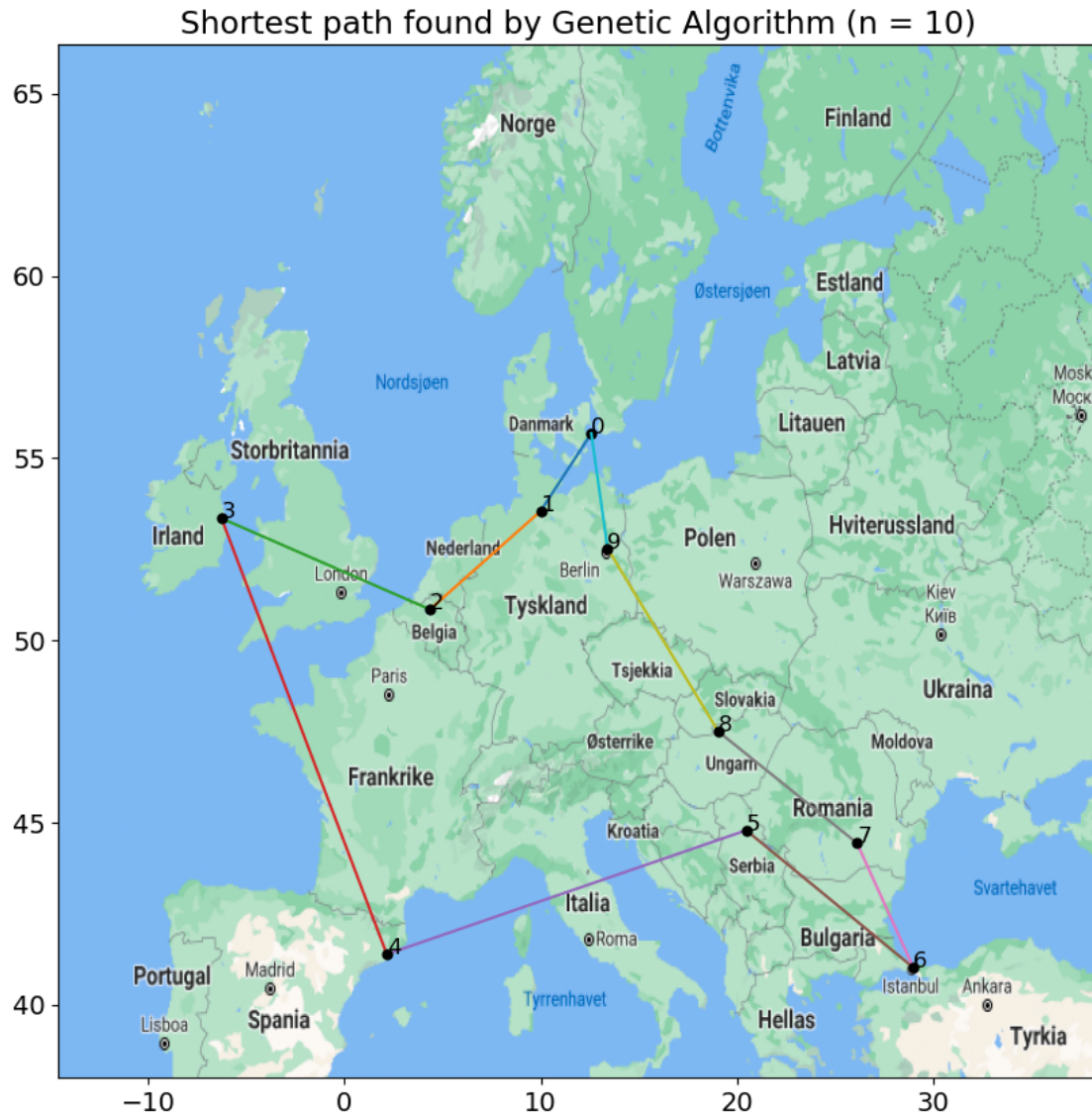
plot_plan(route_shortest.cities, show=False)
plt.title("Shortest path found by Genetic Algorithm (n = 10)")

```

```
plt.show()
```

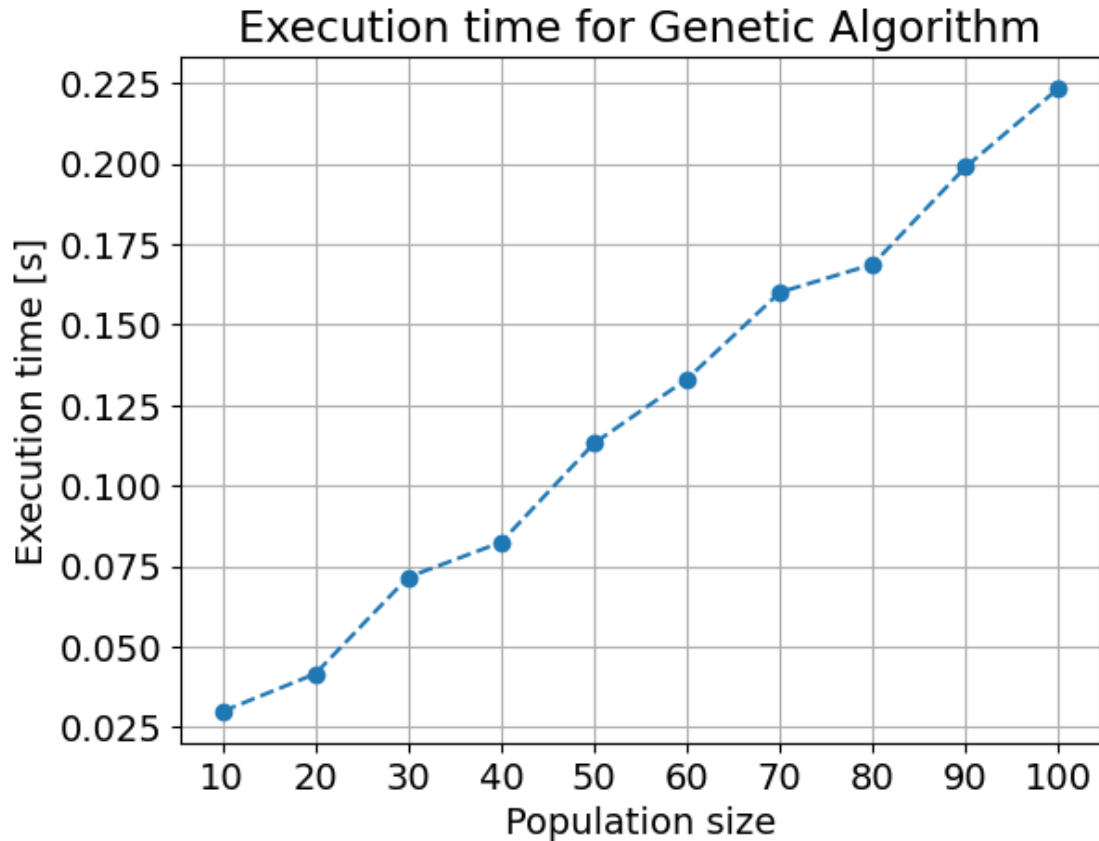
Shortest path: Copenhagen -> Hamburg -> Brussels -> Dublin -> Barcelona ->
Belgrade -> Istanbul -> Bucharest -> Budapest -> Berlin
Shortest distance: 7486 km





Linear time complexity when increasing population size

```
[ ]: exec_times_10 = experiment_result_10.exec_times
plt.plot(population_sizes_10, exec_times_10.mean(axis=1), 'o--')
plt.xlabel("Population size")
plt.xticks(population_sizes_10)
plt.ylabel("Execution time [s]")
plt.title("Execution time for Genetic Algorithm")
plt.grid()
plt.show()
```



1.6.3 24 cites with population sizes 25, 50, and 100

```
[ ]: cities = data[0].copy()
population_sizes = [25, 50, 100]
n_generations = 100

experiment_result_24 = run_ga_experiment(cities, population_sizes,
    ↪n_generations, EGA())
```

100% | 3/3 [00:31<00:00, 10.41s/it]

```
[32]: routes = experiment_result_24.routes
distances = experiment_result_24.distances
histories = experiment_result_24.histories
exec_times = experiment_result_24.exec_times

route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
print(f"Shortest path: {route_shortest}")
print(f"Shortest distance: {route_shortest.distance:.0f} km")
```

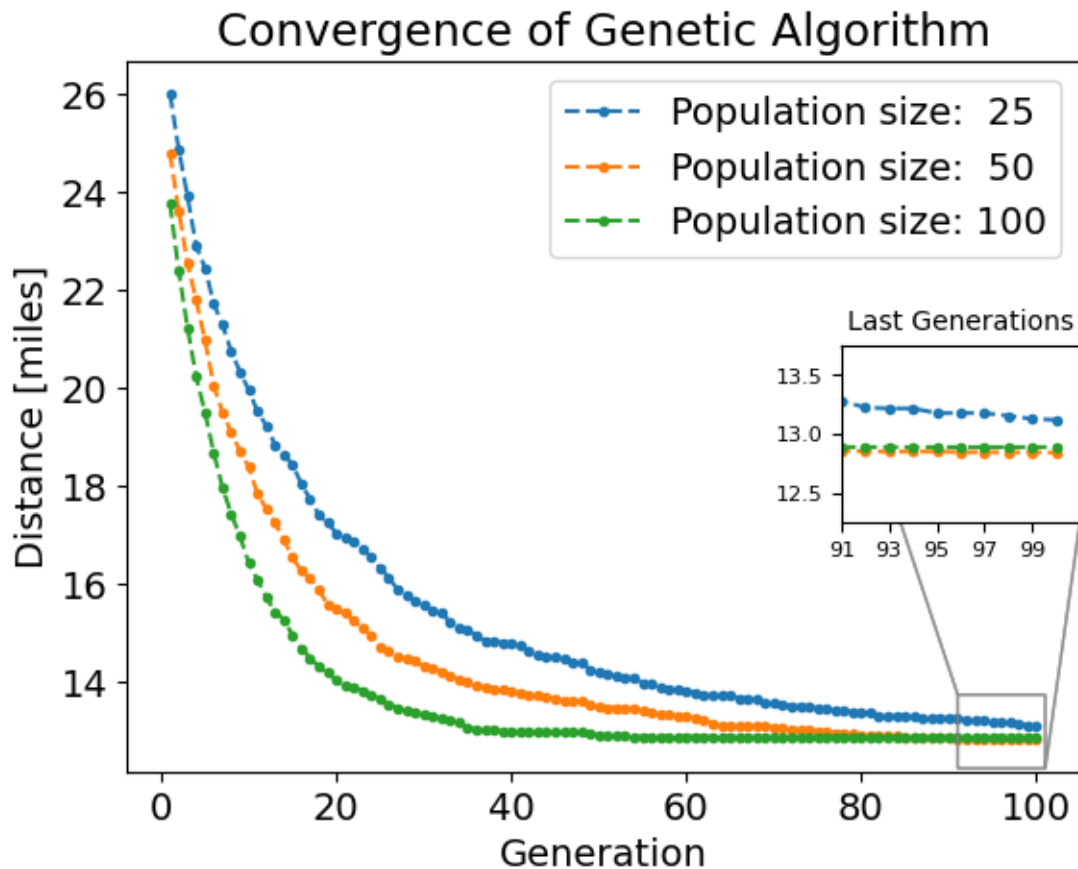
```

plot_convergence(histories, population_sizes, zoom=True)
plot_plan(route_shortest.cities, show=False)
plt.title("Shortest path found by Genetic Algorithm (n = 24)")
plt.show()

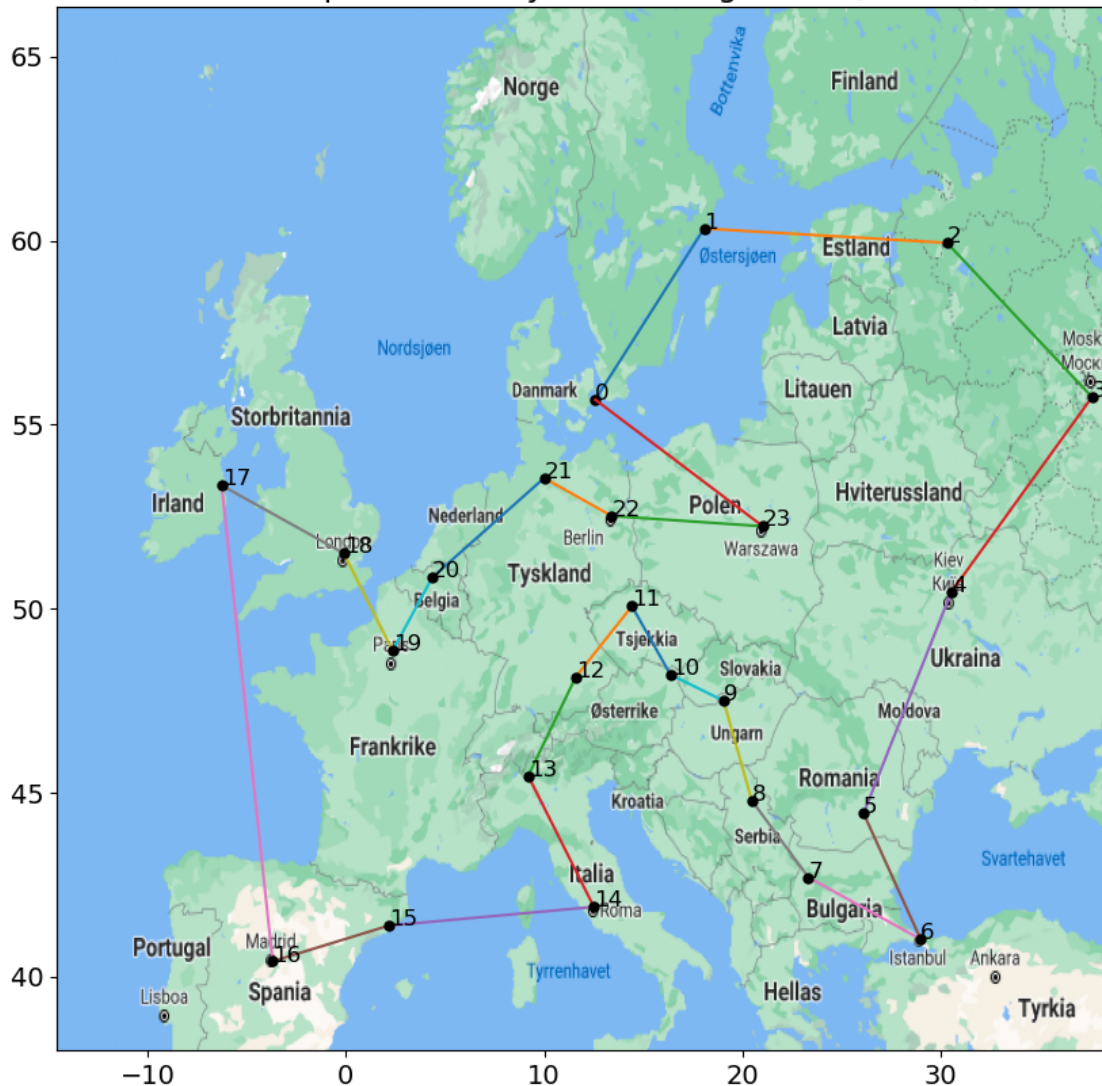
```

Shortest path: Copenhagen -> Stockholm -> Saint Petersburg -> Moscow -> Kyiv -> Bucharest -> Istanbul -> Sofia -> Belgrade -> Budapest -> Vienna -> Prague -> Munich -> Milan -> Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg -> Berlin -> Warsaw

Shortest distance: 12334 km



Shortest path found by Genetic Algorithm (n = 24)



```
[33]: exec_times = experiment_result_24.exec_times
print("Execution times:")
heading = "Population size | Mean time [s] | Std time [s]"
print(heading)
print("-"*len(heading))
for exec_time, pop_size in zip(exec_times, population_sizes):
    mean_time = exec_times.mean()
    std_time = exec_times.std()
    print(f"{pop_size:<15} | {mean_time:<13.2e} | {std_time:<10.2e}")
```

Execution times:

Population size	Mean time [s]	Std time [s]
-----------------	---------------	--------------

25	5.21e-01	2.86e-01
50	5.21e-01	2.86e-01
100	5.21e-01	2.86e-01

1.6.4 10 cities

[]:

Among the first 10 cities, did your GA find the shortest tour (as found by the exhaustive search)? Did it come close? - Yes, it found the shortest tour

For both 10 and 24 cities: How did the running time of your GA compare to that of the exhaustive search? - The GA was significantly faster than the exhaustive search. In the case of 24 cities it beat the exhaustive search by a factor of 10^{15}

How many tours were inspected by your GA as compared to by the exhaustive search?

- Exhaustive search: $24!$ - GA: - Assuming “inspect” means “how many unique tours were made”
 - Assuming no duplicates in the population. - With a population size of 100 and 100 generations and replacing the bottom 10% each generation, the GA inspected $100 + 100 * 0.1 \times 100 = \underline{\underline{1100}}$

1.7 Hybrid Algorithm (IN4050 only)

1.7.1 Lamarckian

Lamarck, 1809: Traits acquired in parents’ lifetimes can be inherited by offspring. In general the algorithms are referred to as Lamarckian if the result of the local search stage replaces the individual in the population.

```
[ ]: class Lamarckian_EGA(EGA):
    """
    Purpose
    -----
    Lamarckian_EGA class for solving the Traveling Salesman Problem (TSP) using
    ↪ a Lamarckian evolutionary approach.

    This class inherits from the EGA (Evolutionary Genetic Algorithm) class and
    ↪ modifies the evaluation
    of the population by replacing each individual with the result of a hill
    ↪ climbing algorithm applied
    to that individual. This approach is based on the Lamarckian principle that
    ↪ traits acquired during
    an individual's lifetime can be inherited by offspring.

    Methods
    -----
    lamarck(population: list[Route]) -> list[Route]
        Replaces the population with the result of a hill climbing algorithm on
    ↪ each member of the population.
    evaluate_population(population: list[Route]) -> list[Route]
```

```

        Evaluates the population based on the fitness of the routes, including
↳ the Lamarckian algorithm
        in the main loop of the `__call__()` method.
        __call__(cls, cities: list[str], population_size: int, n_generations: int)
↳ -> ga_result
        Run the genetic algorithm.
        """

    @classmethod
    def lamarck(cls, population: list[Route]) -> list[Route]:
        """
        Replaces the population with the result of a hill climbing algorithm on
↳ each member of the population.
        """
        population = [hill_climbing(route.cities) for route in population]
        return population

    @classmethod
    def evaluate_population(cls, population):
        """
        Evaluates the population based on the fitness of the routes.
        Altered to include the lamarckian algorithm in the main loop of the
↳ `__call__()` method.
        """
        pop = cls.lamarck(population)
        return cls.fitness_selection(pop)

```

10 cities

```

[ ]: cities = data[0][:10]
    population_sizes = [i*10 for i in range(1, 11)]
    n_generations = 10

    experiment_result_lamarck = run_ga_experiment(cities, population_sizes,
↳ n_generations, Lamarckian_EGA())

```

100% | 10/10 [00:33<00:00, 3.30s/it]

```

[36]: routes = experiment_result_lamarck.routes
    distances = experiment_result_lamarck.distances
    histories = experiment_result_lamarck.histories
    exec_times = experiment_result_lamarck.exec_times

    route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
    print(f"Shortest path: {route_shortest}")
    print(f"Shortest distance: {route_shortest.distance:.0f} km")

    plot_convergence(histories, population_sizes)

```

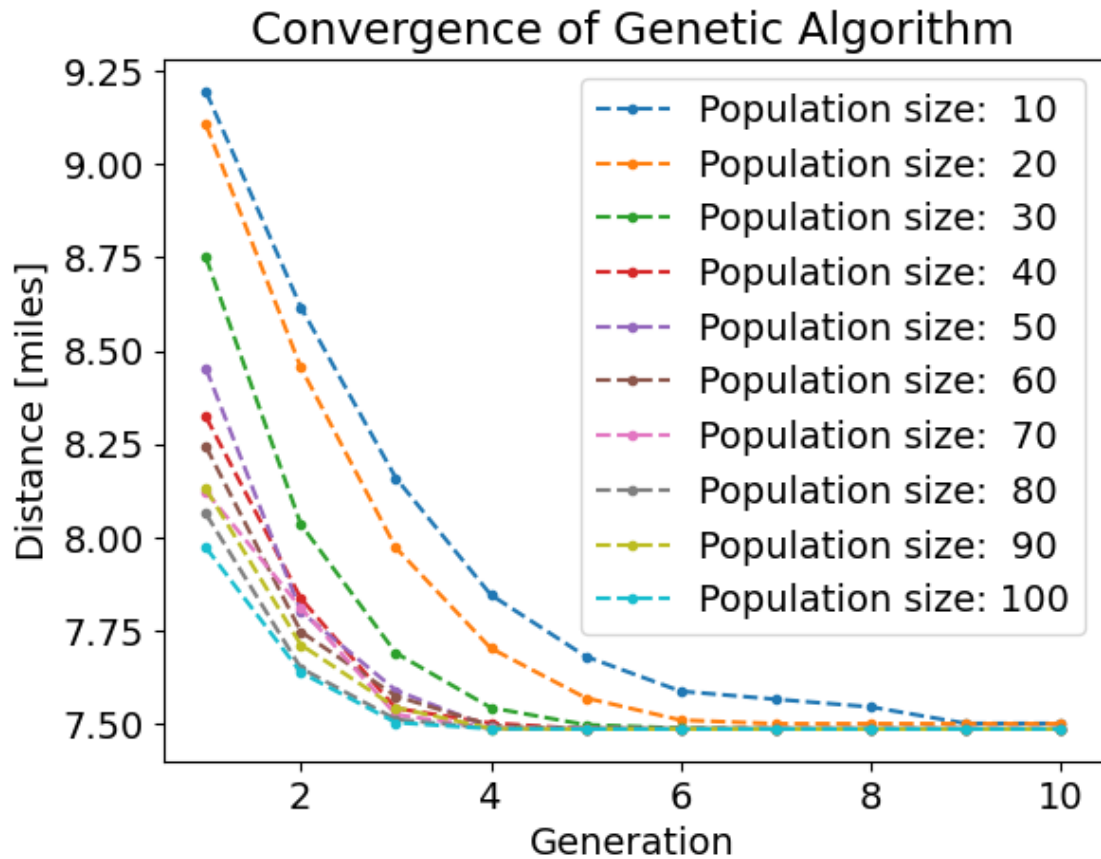


```

plot_plan(route_shortest.cities, show=False)
plt.title("Shortest path found by Lamarckian Hybrid Genetic Algorithm (n = 10)")
plt.show()

```

Shortest path: Berlin -> Copenhagen -> Hamburg -> Brussels -> Dublin ->
 Barcelona -> Belgrade -> Istanbul -> Bucharest -> Budapest
 Shortest distance: 7486 km



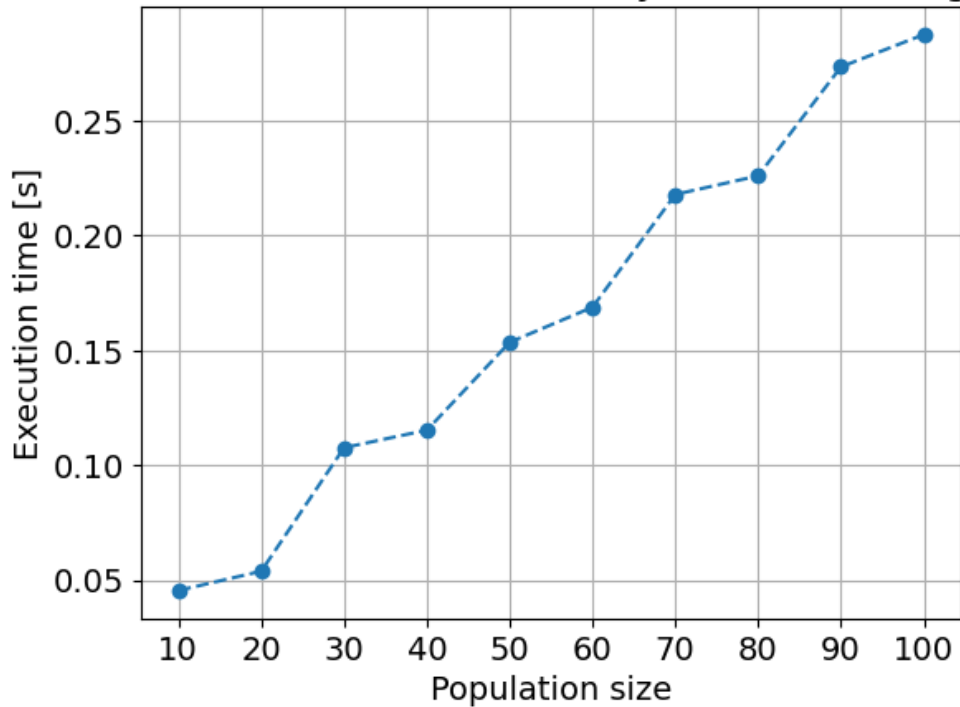
Shortest path found by Lamarckian Hybrid Genetic Algorithm (n = 10)



Linear time complexity when increasing population size

```
[ ]: exec_times_lamarck = experiment_result_lamarck.exec_times
plt.plot(population_sizes, exec_times_lamarck.mean(axis=1), 'o--')
plt.xlabel("Population size")
plt.xticks(population_sizes)
plt.ylabel("Execution time [s]")
plt.title("Execution time for Lamarckian Hybrid Genetic Algorithm")
plt.grid()
plt.show()
```

Execution time for Lamarckian Hybrid Genetic Algorithm



24 cities

```
[ ]: cities = data[0].copy()
     population_sizes = [25, 50, 100]
     n_generations = 20

     experiment_result_lamarck_24 = run_ga_experiment(cities, population_sizes, n_generations, Lamarckian_EGA())
```

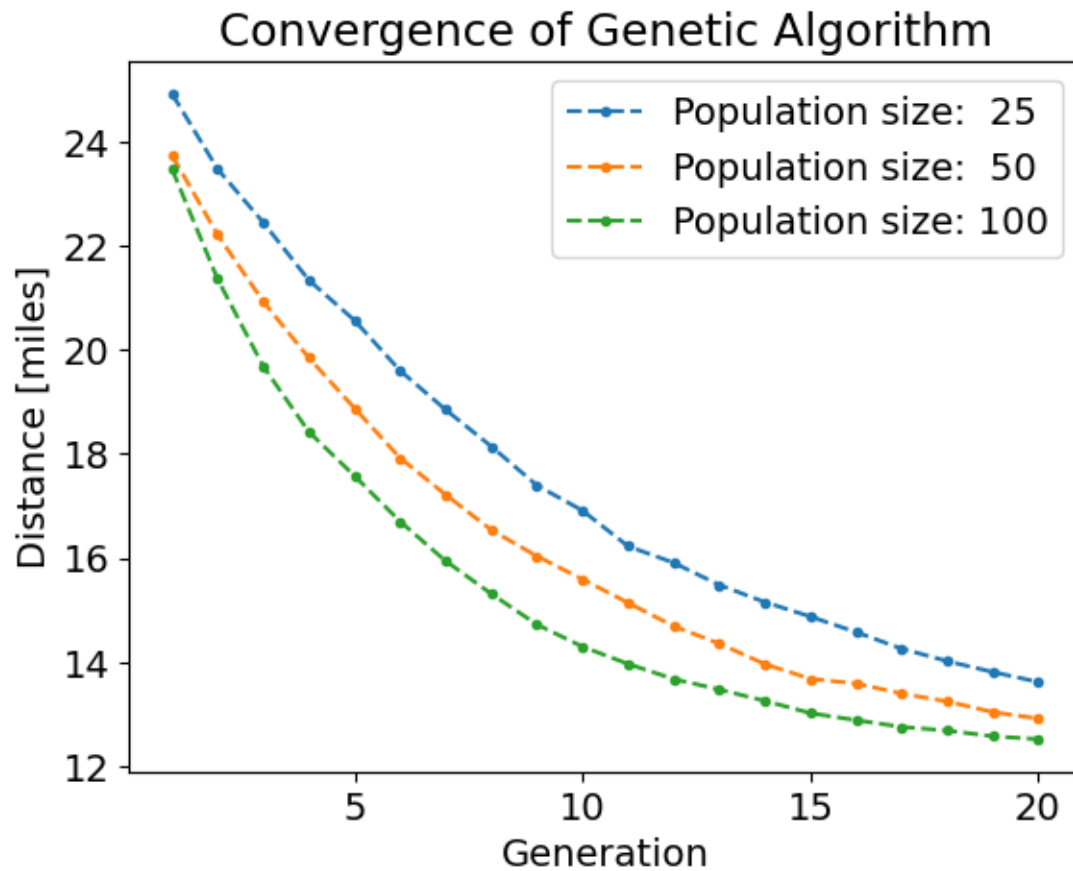
100% | 3/3 [02:30<00:00, 50.24s/it]

```
[39]: routes = experiment_result_lamarck_24.routes
     distances = experiment_result_lamarck_24.distances
     histories = experiment_result_lamarck_24.histories
     exec_times = experiment_result_lamarck_24.exec_times

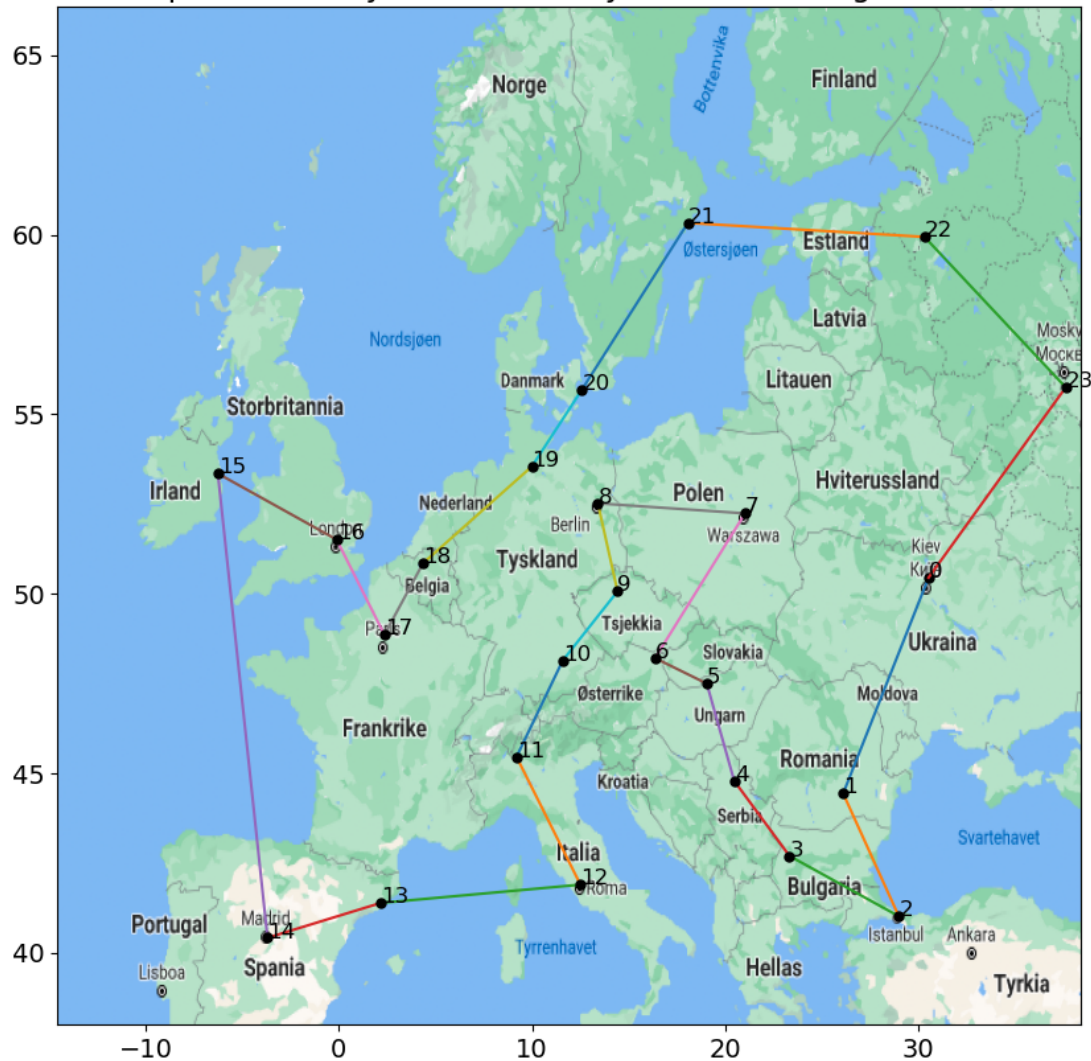
     route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
     print(f"Shortest path: {route_shortest}")
     print(f"Shortest distance: {route_shortest.distance:.0f} km")

     plot_convergence(histories, population_sizes, zoom=False)
     plot_plan(route_shortest.cities, show=False)
     plt.title("Shortest path found by Lamarckian Hybrid Genetic Algorithm (n = 24)")
     plt.show()
```

Shortest path: Kyiv -> Bucharest -> Istanbul -> Sofia -> Belgrade -> Budapest -> Vienna -> Warsaw -> Berlin -> Prague -> Munich -> Milan -> Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg -> Copenhagen -> Stockholm -> Saint Petersburg -> Moscow
Shortest distance: 12287 km



Shortest path found by Lamarckian Hybrid Genetic Algorithm (n = 24)



1.7.2 Baldwinian

Baldwin effect suggests a mechanism whereby evolutionary progress can be guided towards favourable adaptation without the changes in individual's fitness arising from learning or development being reflected in changed genetic characteristics. In general the algorithms are referred to as Baldwinian if the original member is kept, but has as its fitness the value belonging to the outcome of the local search process.

(See chapter 10 and 10.2.1 from Eiben and Smith textbook for more details. It will also be lectured in Lecure 3)

```
[ ]: class Baldwinian_EGA(EGA):
    """
    Purpose
```

```

-----
    Baldwinian_EGA class for solving the Traveling Salesman Problem (TSP) using
    ↪ a Baldwinian evolutionary approach.

```

```

    This class inherits from the EGA (Evolutionary Genetic Algorithm) class and
    ↪ modifies the evaluation
    of the population by assigning a potential fitness to each individual based
    ↪ on the result of a hill
    climbing algorithm applied to that individual. This approach is based on
    ↪ the Baldwinian principle
    that evolutionary progress can be guided towards favorable adaptation
    ↪ without the changes in an
    individual's fitness arising from learning or development being reflected
    ↪ in changed genetic characteristics.

```

Methods

```

-----
    tournament_selection(population: list[Route], n: int) -> list[Route]
        Select the best route in terms of potential fitness from a random
    ↪ sample of the population.
    potential_fitness_selection(population: list[Route]) -> list[Route]
        Sort the population based on the potential fitness of the routes.
    baldwinian(population: list[Route]) -> list[Route]
        Finds the potential fitness of each route in the population.
    evaluate_population(population: list[Route]) -> list[Route]
        Evaluate the population based on the potential fitness of the routes.
    __call__(cls, cities: list[str], population_size: int, n_generations: int)
    ↪ -> ga_result
        Run the genetic algorithm.
    """

    @classmethod
    def tournament_selection(cls, population: list[Route], n: int) ->
    ↪ list[Route]:
        """
        Select the best route in terms of potential fitness from a random
    ↪ sample of the population.
        """
        return max(sample(population, n), key=lambda route: route.
    ↪ potential_fitness)

    @classmethod
    def potential_fitness_selection(cls, population: list[Route]) ->
    ↪ list[Route]:
        """
        Sort the population based on the potential fitness of the routes.

```

```

    """
    return sorted(population, key=lambda route: route.potential_fitness,
↪reverse=True)

    @classmethod
    def baldwinian(cls, population: list[Route]) -> list[Route]:
        """
        Finds the potential fitness of each route in the population.
        """
        new_population = []
        for route in population:
            potential_route = hill_climbing(route.cities)
            route.potential_fitness = potential_route.fitness
            new_population.append(route)
        return new_population

    @classmethod
    def evaluate_population(cls, population: list[Route]) -> list[Route]:
        """
        Evaluate the population based on the potential fitness of the routes.
        Altered to include the baldwinian algorithm in the main loop of the
↪`__call__()` method.
        """
        population = cls.baldwinian(population)
        return cls.potential_fitness_selection(population)

```

10 cities

```

[ ]: cities = data[0][:10]
    population_sizes = [i*10 for i in range(1, 11)]
    n_generations = 10

    experiment_result_baldwinian = run_ga_experiment(cities, population_sizes,
↪n_generations, Baldwinian_EGA())

```

```

0%|          | 0/10 [00:00<?, ?it/s]

100%|        | 10/10 [00:22<00:00, 2.27s/it]

```

```

[42]: routes = experiment_result_baldwinian.routes
    distances = experiment_result_baldwinian.distances
    histories = experiment_result_baldwinian.histories
    exec_times = experiment_result_baldwinian.exec_times

    route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
    print(f"Shortest path: {route_shortest}")
    print(f"Shortest distance: {route_shortest.distance:.0f} km")

    fig = plot_convergence(histories, population_sizes, show=False)

```

```

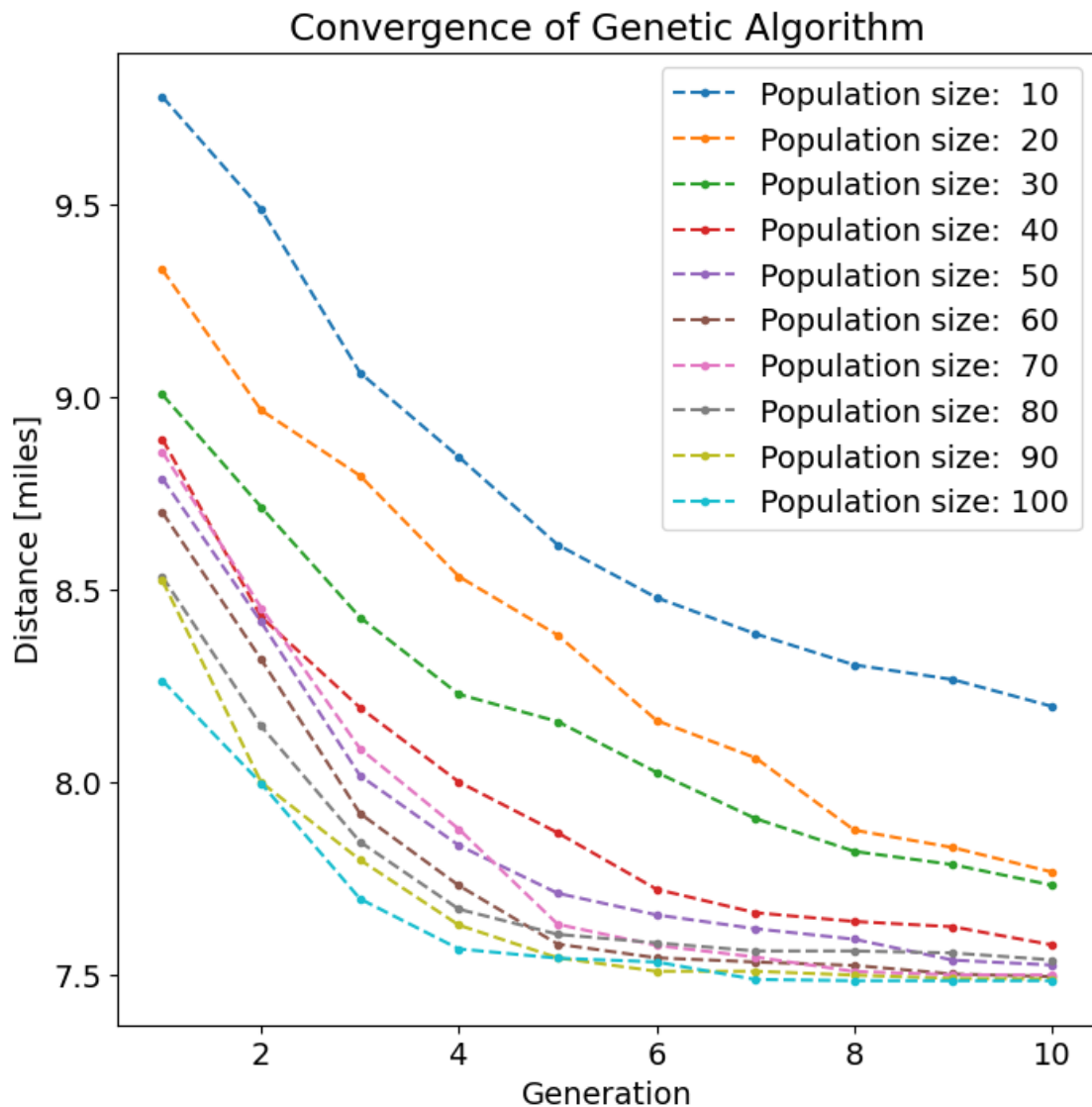
fig.set_size_inches(8, 8)
plt.show()

plot_plan(route_shortest.cities, show=False)
plt.title("Shortest path found by Baldwinian Hybrid Genetic Algorithm (n = 10)")
plt.show()

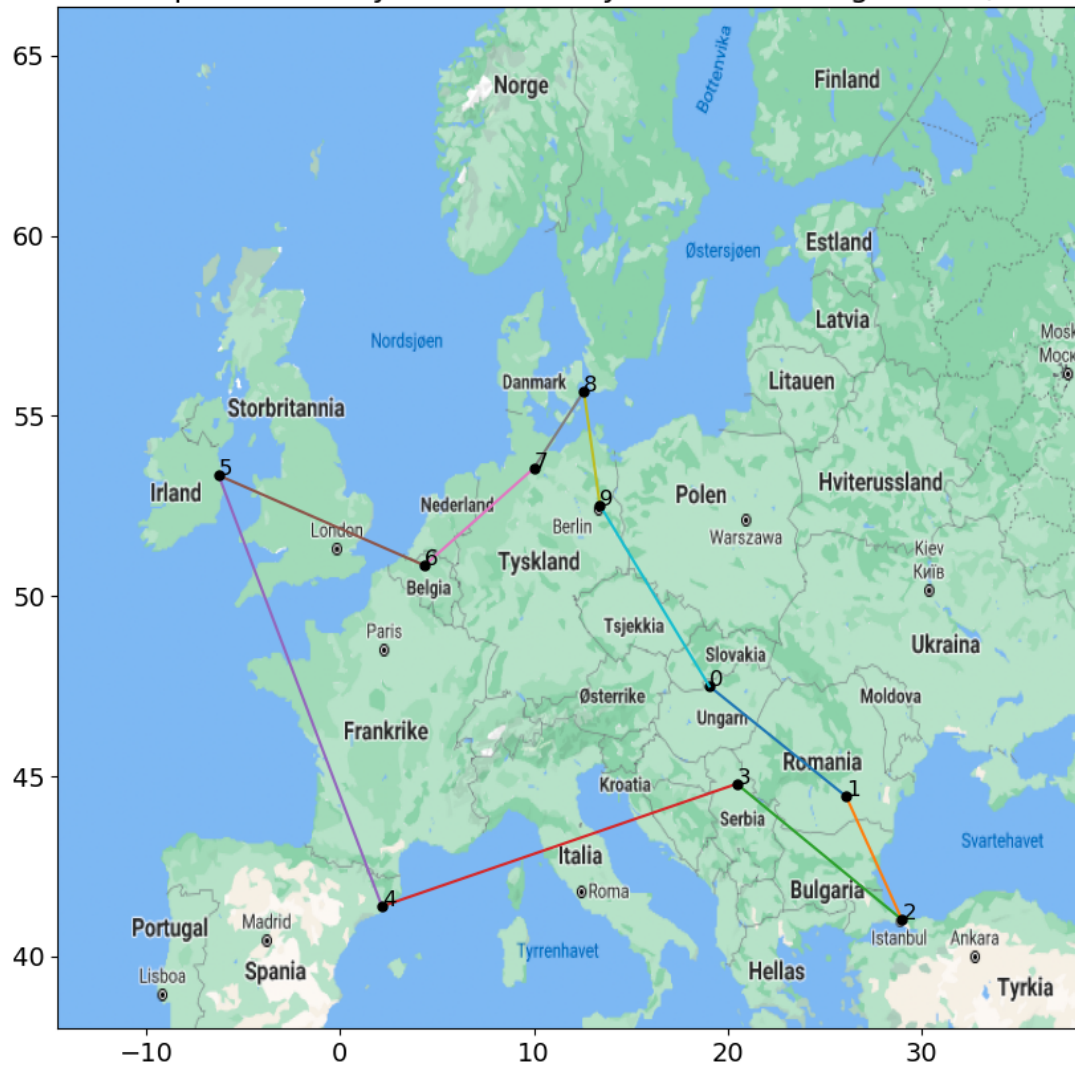
```

Shortest path: Budapest -> Bucharest -> Istanbul -> Belgrade -> Barcelona -> Dublin -> Brussels -> Hamburg -> Copenhagen -> Berlin

Shortest distance: 7486 km



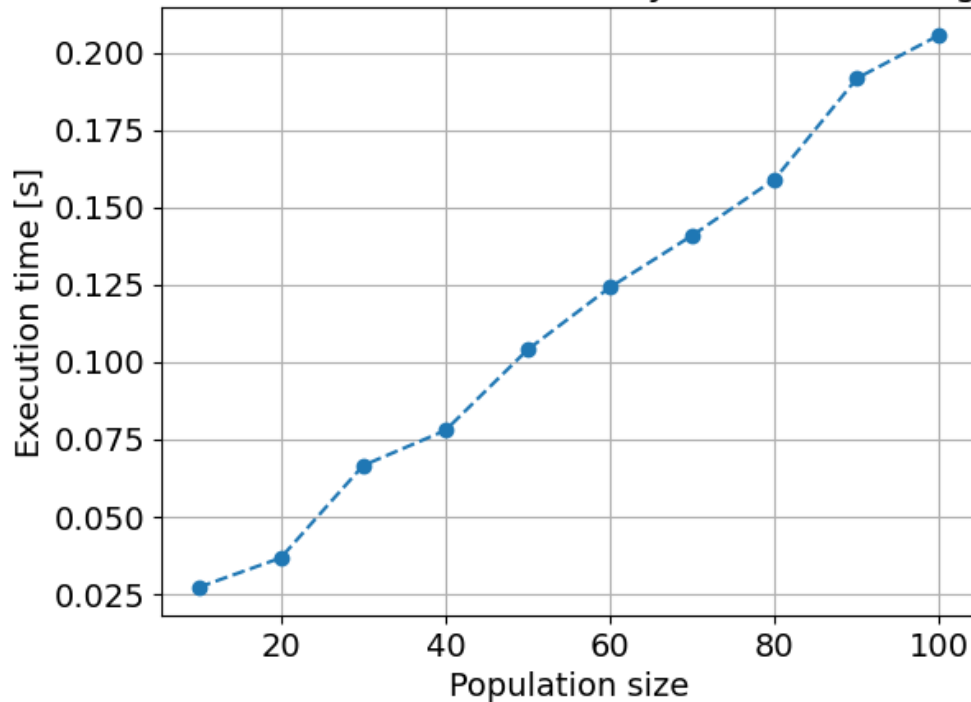
Shortest path found by Baldwinian Hybrid Genetic Algorithm (n = 10)



Linear time complexity when increasing population size

```
[ ]: exec_times_baldwinian = experiment_result_baldwinian.exec_times
plt.plot(population_sizes, exec_times_baldwinian.mean(axis=1), 'o--')
plt.xlabel("Population size")
plt.ylabel("Execution time [s]")
plt.title("Execution time for Baldwinian Hybrid Genetic Algorithm")
plt.grid()
plt.show()
```

Execution time for Baldwinian Hybrid Genetic Algorithm



24 cities

```
[ ]: cities = data[0].copy()
    population_sizes = [25, 50, 100]
    n_generations = 40

    experiment_result_baldwinian_24 = run_ga_experiment(cities, population_sizes, n_generations, Baldwinian_EGA())
```

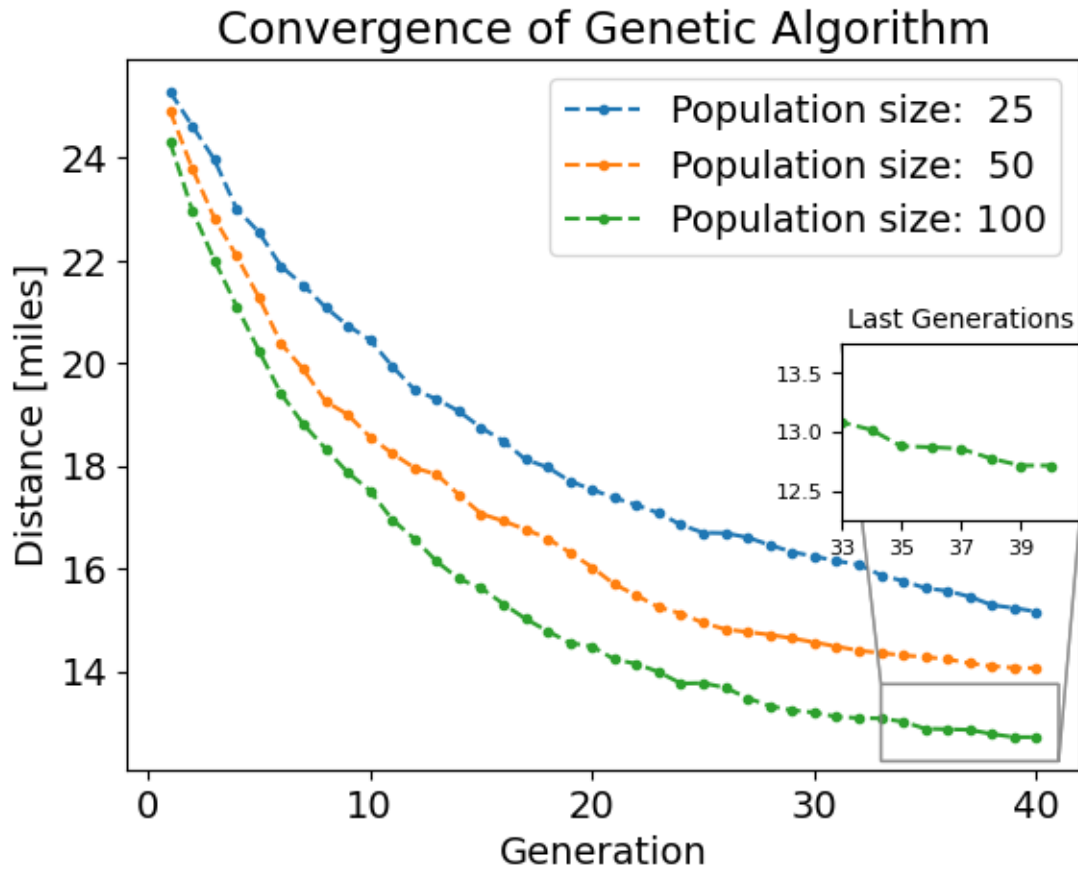
100% | 3/3 [03:00<00:00, 60.20s/it]

```
[45]: routes = experiment_result_baldwinian_24.routes
    distances = experiment_result_baldwinian_24.distances
    histories = experiment_result_baldwinian_24.histories
    exec_times = experiment_result_baldwinian_24.exec_times

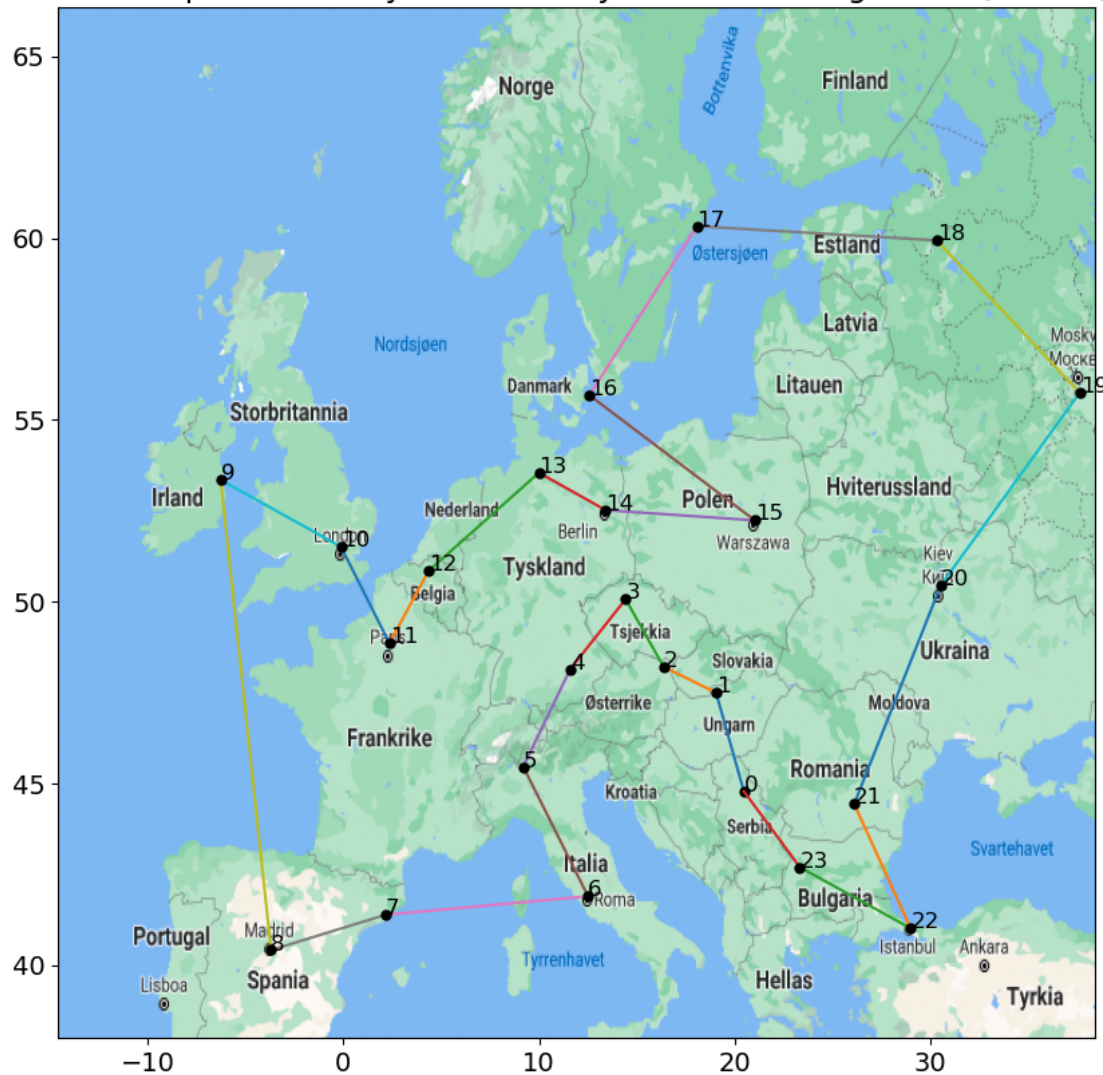
    route_shortest = sorted(routes[-1], key=lambda route: route.distance)[0]
    print(f"Shortest path: {route_shortest}")
    print(f"Shortest distance: {route_shortest.distance:.0f} km")

    plot_convergence(histories, population_sizes, zoom=True)
    plot_plan(route_shortest.cities, show=False)
    plt.title("Shortest path found by Baldwinian Hybrid Genetic Algorithm (n = 24)")
    plt.show()
```

Shortest path: Belgrade -> Budapest -> Vienna -> Prague -> Munich -> Milan ->
Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg
-> Berlin -> Warsaw -> Copenhagen -> Stockholm -> Saint Petersburg -> Moscow ->
Kyiv -> Bucharest -> Istanbul -> Sofia
Shortest distance: 12334 km



Shortest path found by Baldwinian Hybrid Genetic Algorithm (n = 24)



1.7.3 Task

Implement a hybrid algorithm to solve the TSP: Couple your GA and hill climber by running the hill climber a number of iterations on each individual in the population as part of the evaluation. Test both Lamarckian and Baldwinian learning models and report the results of both variants in the same way as with the pure GA (min, max, mean and standard deviation of the end result and an averaged generational plot). How do the results compare to that of the pure GA, considering the number of evaluations done?

1.8 Summary

- As seen below, **all solutions found the shortest or near shortest** within a few hundred km depending on the run (varies between runs).

- Time complexity of all the GA is **linear** with respect to population size.
- The regular GA is **one order of magnitude faster** than both the Lamarckian and Baldwinian GA.
- The Lamarckian GA reaches the **same result** as the Baldwinian GA, but **in half the number of generations**. Both methods are approximately the same speed, but the Lamarckian GA has twice the standard deviation, meaning the speed is more inconsistent.
- All in all, the **regular GA** is the **fastest** which is the most important factor in solving the problem. My Hill Climber is not performing that well, which is likely to be the reason there is no payoff for the slower execution time of the Lamarckian and Baldwinian GA.

```
[ ]: benchmark_evo_alg(experiment_result_24)
```

Results

Shortest path: Copenhagen -> Stockholm -> Saint Petersburg -> Moscow -> Kyiv -> Bucharest -> Istanbul -> Sofia -> Belgrade -> Budapest -> Vienna -> Prague -> Munich -> Milan -> Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg -> Berlin -> Warsaw

Longest path: Kyiv -> Moscow -> Saint Petersburg -> Stockholm -> Copenhagen -> Hamburg -> London -> Dublin -> Madrid -> Barcelona -> Paris -> Brussels -> Berlin -> Prague -> Budapest -> Vienna -> Munich -> Milan -> Rome -> Belgrade -> Sofia -> Istanbul -> Bucharest -> Warsaw

Shortest distance: 12334 km
 Longest distance: 13524 km
 Mean distance: 12891 ± 339 km

Shortest execution time: 8.7e-01 seconds
 Longest execution time: 1.1e+00 seconds
 Mean execution time: 9.0e-01 ± 3.7e-02 seconds

```
[47]: benchmark_evo_alg(experiment_result_lamarck_24)
```

Results

Shortest path: Kyiv -> Bucharest -> Istanbul -> Sofia -> Belgrade -> Budapest -> Vienna -> Warsaw -> Berlin -> Prague -> Munich -> Milan -> Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg -> Copenhagen -> Stockholm -> Saint Petersburg -> Moscow

Longest path: Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg -> Prague -> Berlin -> Copenhagen -> Stockholm -> Saint Petersburg -> Moscow -> Kyiv -> Warsaw -> Budapest -> Bucharest -> Istanbul -> Sofia -> Belgrade -> Vienna -> Munich -> Milan -> Rome

Shortest distance: 12287 km
 Longest distance: 12913 km
 Mean distance: 12519 ± 174 km

Shortest execution time: 3.5e+00 seconds
Longest execution time: 5.3e+00 seconds
Mean execution time: 4.3e+00 ± 5.4e-01 seconds

[48]: `benchmark_evo_alg(experiment_result_baldwinian_24)`

Results

Shortest path: Belgrade -> Budapest -> Vienna -> Prague -> Munich -> Milan ->
Rome -> Barcelona -> Madrid -> Dublin -> London -> Paris -> Brussels -> Hamburg
-> Berlin -> Warsaw -> Copenhagen -> Stockholm -> Saint Petersburg -> Moscow ->
Kyiv -> Bucharest -> Istanbul -> Sofia

Longest path: Prague -> Hamburg -> Brussels -> Dublin -> London -> Paris ->
Milan -> Madrid -> Barcelona -> Rome -> Belgrade -> Sofia -> Istanbul ->
Bucharest -> Kyiv -> Moscow -> Saint Petersburg -> Stockholm -> Copenhagen ->
Berlin -> Warsaw -> Budapest -> Vienna -> Munich

Shortest distance: 12334 km
Longest distance: 13379 km
Mean distance: 12718 ± 263 km

Shortest execution time: 3.2e+00 seconds
Longest execution time: 7.0e+00 seconds
Mean execution time: 4.9e+00 ± 1.2e+00 seconds